

---

# Object Detection Underneath Sparse Occlusion of Forests

---

*A thesis submitted in fulfilment of the requirements  
for the degree of Master of Technology*

*by*

Nitik Jain

17807448



DEPARTMENT OF AEROSPACE ENGINEERING  
INDIAN INSTITUTE OF TECHNOLOGY KANPUR

December 2022

## Certificate

It is certified that the work contained in this thesis entitled “Object Detection Underneath Sparse Occlusion of Forests” by “Nitik Jain” has been carried out under my supervision and that it has not been submitted elsewhere for a degree.



Dr. Mangal Kothari

Professor

Department of Aerospace Engineering  
Indian Institute of Technology Kanpur

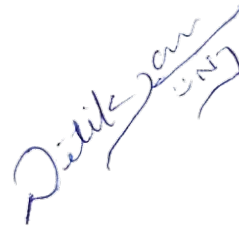
*December 2022*



# Declaration

This is to certify that the thesis titled “Object Detection Underneath Sparse Occlusion of Forests” has been authored by me. It presents the research conducted by me under the supervision of “Dr. Mangal Kothari”.

To the best of my knowledge, it is an original work, both in terms of research content and narrative, and has not been submitted elsewhere, in part or in full, for a degree. Further, due credit has been attributed to the relevant state-of-the-art and collaborations (if any) with appropriate citations and acknowledgements, in line with established norms and practices.



Nitik Jain

*December 2022*

Department of Aerospace Engineering  
Indian Institute of Technology Kanpur

# *Abstract*

---

Name of the student: **Nitik Jain**

Roll No: **17807448**

Degree for which submitted: **M.Tech.**

Department: **Aerospace Engineering**

Thesis title: **Object Detection Underneath Sparse Occlusion of Forests**

Thesis supervisor: **Dr. Mangal Kothari**

Month and year of thesis submission: **December 2022**

---

Occlusion of object due to vegetation is a crucial challenge for remote sensing applications in many different domains, including but not limited to search and rescue, surveillance, and control of borders, among other domains. Detection of objects under the dense canopy of forests enables numerous advantages to the authority carrying the operation. In particular, Search and Rescue (SAR) operations are of utmost importance and need technological advancements.

This work methodically aims to develop an end-to-end pipeline for enhancing the detection of objects (people in particular) beneath the partial occlusion of a forest canopy. This study establishes a framework for image acquisition from multiple modalities, including IR cameras, RGB cameras, and LiDAR. Initially, LiDAR experiments were done to examine the potential of LiDAR data to penetrate canopy. These experiments demonstrated the restricted penetrability of LiDARs in such an environment. Then, a variety of fusion algorithms were evaluated, and the multi-modality image fusion framework MST-SR was identified and implemented to fuse the IR and RGB modalities. In addition, the study investigated several synthetic aperture imaging techniques with the intent of increasing the visibility of objects occluded by the vegetative canopy. The principles of Dynamically Reparameterized Light Field modelling is used to create integral images using Airborne



Optical Sectioning (AOS), which is a wide synthetic-aperture imaging technique. The hypothesis has been validated by performing AOS on synthetic simulated data where enhanced visibility of partially occluded humans is observed. Lastly, YOLOv5 has been re-trained and fine tuned on thermal images from Teledyne FLIR thermal dataset to perform detection on images from fused modalities. A mAP of 0.83 was achieved on top 3 classes present in the test set.

This work in this research can be considered as the initial step toward our ultimate objective of developing a fully functional real-time person detection and tracking system to detect humans in forests. It established the proof-of-concept results, and as it proceeds toward implementation in the actual world, more representative accuracy is anticipated.

## *Acknowledgements*

I would like to express my sincere gratitude to my thesis advisor, Dr. Mangal Kothari, without whose unwavering support throughout my undergraduate studies, I would be a mere shadow of who I am today. My regular interactions with him gave me a tremendous deal of clarity on various aspects of this work and life in general. His moral support after each dismal failure helped me persevere through the tough moments. He gave me the freedom to work at my own pace, encouraged new ideas, and assisted me in developing a research rationale. This work marks my four year long journey of working under the tutelage of sir.

Throughout this work, I was fortunate enough to receive help at numerous instances. I thank Prof. Abhishek, Prof. Salil Goel, and Prof. K S Venkatesh for their valuable inputs in shaping the work and their assistance in procuring resources to carry out the research. I would also like to thank everyone at the wonderful research community at IGCL, with whom I spend most of the time in lab. Special thanks to my friend Gavish Jain for persevering until the end.

I must also express my very profound gratitude to my parents and grandparents for providing me with unfailing support and continuous encouragement throughout my years of study. This accomplishment would not have been possible without them, and I am incredibly indebted to them.

# Contents

|                         |             |
|-------------------------|-------------|
| <b>Acknowledgements</b> | <b>vi</b>   |
| <b>List of Figures</b>  | <b>x</b>    |
| <b>List of Tables</b>   | <b>xiii</b> |
| <b>Abbreviations</b>    | <b>xiv</b>  |

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>1 Introduction</b>                                                       | <b>1</b>  |
| 1.1 Motivation . . . . .                                                    | 1         |
| 1.2 The Problem Statement . . . . .                                         | 2         |
| 1.3 Outline of Thesis . . . . .                                             | 3         |
| <b>2 Background</b>                                                         | <b>4</b>  |
| 2.1 Establishing Occlusion and its context . . . . .                        | 4         |
| 2.2 The Problem of Recognizing Objects in Images . . . . .                  | 6         |
| 2.3 Neural Networks . . . . .                                               | 9         |
| 2.3.1 Perceptrons . . . . .                                                 | 9         |
| 2.3.2 Convolution . . . . .                                                 | 10        |
| 2.3.3 Training a Neural Network . . . . .                                   | 11        |
| 2.3.4 Parallel between Control Theory and Neural Network training . . . . . | 12        |
| 2.3.5 Convolution Neural Networks . . . . .                                 | 14        |
| 2.3.6 Loss Functions and Activation Functions . . . . .                     | 17        |
| 2.3.6.1 Classification Loss . . . . .                                       | 17        |
| 2.3.6.2 Regression Loss . . . . .                                           | 17        |
| 2.3.6.3 Activation Functions . . . . .                                      | 18        |
| <b>3 Description of Sensors, Dataset and Simulation Environment</b>         | <b>20</b> |
| 3.1 Active and Passive Sensors . . . . .                                    | 20        |
| 3.2 RGB and Thermal Camera . . . . .                                        | 22        |
| 3.2.1 Pinhole Camera Modeling . . . . .                                     | 23        |
| 3.3 Laser Scanner - FARO Focus Premium 350 . . . . .                        | 24        |
| 3.4 Training Dataset . . . . .                                              | 25        |

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| 3.4.1    | Overview of COCO Dataset . . . . .                                                    | 26        |
| 3.4.2    | Overview of Teledyne FLIR Thermal Dataset . . . . .                                   | 28        |
| 3.5      | The Simulation Environment . . . . .                                                  | 29        |
| <b>4</b> | <b>Image Acquisition - Synthetic Aperture Imaging and Airborne Optical Sectioning</b> | <b>32</b> |
| 4.1      | Camera aperture and its relation to Depth of Field . . . . .                          | 32        |
| 4.2      | Image based Rendering . . . . .                                                       | 33        |
| 4.3      | Light Field Rendering . . . . .                                                       | 34        |
| 4.3.1    | 4D Light Field . . . . .                                                              | 35        |
| 4.3.2    | Dynamically Reparameterized Light Field . . . . .                                     | 37        |
| 4.3.3    | Variable Aperture . . . . .                                                           | 38        |
| 4.4      | Airborne Optical Sectioning . . . . .                                                 | 39        |
| 4.4.1    | Synthetic Aperture Imaging . . . . .                                                  | 39        |
| 4.4.2    | Airborne Optical Sectioning (AOS) . . . . .                                           | 39        |
| 4.4.3    | Light Field Renderer Module . . . . .                                                 | 41        |
| <b>5</b> | <b>Image Fusion and Object Detection</b>                                              | <b>42</b> |
| 5.1      | Visible and Thermal Image Fusion . . . . .                                            | 42        |
| 5.1.1    | VIFB . . . . .                                                                        | 43        |
| 5.1.2    | Dataset . . . . .                                                                     | 43        |
| 5.1.3    | Fusion Algorithms . . . . .                                                           | 44        |
| 5.1.4    | Benchmarking Metrics . . . . .                                                        | 44        |
| 5.2      | Multi-scale Transform and Sparse Representation Fusion . . . . .                      | 45        |
| 5.3      | Evaluation Metrics . . . . .                                                          | 48        |
| 5.3.1    | Chen-Blum Metric . . . . .                                                            | 48        |
| 5.3.2    | Chen-Varshney Metric . . . . .                                                        | 48        |
| 5.4      | Object Detection Network . . . . .                                                    | 49        |
| 5.4.1    | Transfer Learning Paradigm . . . . .                                                  | 49        |
| 5.4.2    | Region Proposal Networks . . . . .                                                    | 50        |
| 5.4.3    | Anchor Boxes . . . . .                                                                | 51        |
| 5.4.4    | Intersection over Union (IoU) . . . . .                                               | 53        |
| 5.4.5    | Non-Maximum Suppression . . . . .                                                     | 54        |
| 5.4.6    | Evaluation Metrics . . . . .                                                          | 55        |
| 5.4.6.1  | Confusion Matrix . . . . .                                                            | 56        |
| 5.4.6.2  | Precision and Recall . . . . .                                                        | 56        |
| 5.4.6.3  | Mean Average Precision (mAP) . . . . .                                                | 57        |
| 5.4.7    | You Only Look Once . . . . .                                                          | 57        |
| 5.4.7.1  | Unified Detection . . . . .                                                           | 57        |
| 5.4.7.2  | Model Architecture . . . . .                                                          | 59        |
| <b>6</b> | <b>Methodology and Results</b>                                                        | <b>61</b> |
| 6.1      | Experiments with Laser Scanner . . . . .                                              | 62        |
| 6.1.1    | LiDAR configuration . . . . .                                                         | 62        |
| 6.1.2    | Experiment 1 . . . . .                                                                | 63        |

---

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| 6.1.2.1  | Observations . . . . .                    | 64        |
| 6.1.3    | Experiment 2 . . . . .                    | 65        |
| 6.1.3.1  | Observations . . . . .                    | 66        |
| 6.2      | RGB and Thermal Fusion . . . . .          | 67        |
| 6.2.1    | Results on VIFB images . . . . .          | 67        |
| 6.2.2    | Results on self-acquired images . . . . . | 68        |
| 6.2.2.1  | Scenario 1 . . . . .                      | 69        |
| 6.2.2.2  | Scenario 2 . . . . .                      | 69        |
| 6.3      | Object Detection using YOLOv5 . . . . .   | 70        |
| 6.3.1    | EDA on Teledyne FLIR Dataset . . . . .    | 70        |
| 6.3.2    | Inference and Results . . . . .           | 71        |
| 6.4      | AOS on Simulation Data . . . . .          | 73        |
| <b>7</b> | <b>Conclusion</b>                         | <b>77</b> |
| 7.0.1    | Contributions of the Work . . . . .       | 77        |
| 7.0.2    | Limitations of the Work . . . . .         | 79        |
| 7.0.3    | Future Prospects . . . . .                | 79        |

## Bibliography

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                        |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Flow chart depicting the blocks of pipeline . . . . .                                                                                                                                                                                                                                                                  | 2  |
| 2.1 | (a) An example of occlusion due to one object being hidden partially behind other object, in this case the mesh hiding objects behind. (b) An example of occlusion due to sensor configuration. . . . .                                                                                                                | 5  |
| 2.2 | Building structure captured beneath tree canopy. Image: VeriDaaS, used with permission.[1] . . . . .                                                                                                                                                                                                                   | 6  |
| 2.3 | A montage describing the difference the difference of various object recognition techniques. . . . .                                                                                                                                                                                                                   | 8  |
| 2.4 | (a) Pictorial representation of a perceptron. (b) A pictorial representation of how single perceptrons make a MLP network. . . . .                                                                                                                                                                                     | 9  |
| 2.5 | A traditional control system . . . . .                                                                                                                                                                                                                                                                                 | 13 |
| 2.6 | A diagram drawing comparisons between control system and neural network                                                                                                                                                                                                                                                | 14 |
| 2.7 | Overview of CNN learning features and finally classifying for the given task                                                                                                                                                                                                                                           | 14 |
| 2.8 | Depiction of various layers in a CNN with reduction of output vector . . . .                                                                                                                                                                                                                                           | 16 |
| 2.9 | Tabular representation of various activation function with their graphs, mathematical representation and derivatives. . . . .                                                                                                                                                                                          | 19 |
| 3.1 | Image shows the range of various sensors used in remote sensing and along with the vehicle they are mounted on. This work primarily entails to ground-based region (<150m) and hence LiDAR and Optical Sensors are shortlisted keeping in mind their compatibility with UAV. . . . .                                   | 21 |
| 3.2 | The CAT S60 smartphone with locations of IR and RGB sensors . . . . .                                                                                                                                                                                                                                                  | 22 |
| 3.3 | A pictorial representation of pinhole camera model . . . . .                                                                                                                                                                                                                                                           | 23 |
| 3.4 | Some sample images from COCO dataset visualizes in the interface of Fifty-One . . . . .                                                                                                                                                                                                                                | 27 |
| 3.5 | Some frames from the Teledyne FLIR ADAS dataset for thermal algorithmic training . . . . .                                                                                                                                                                                                                             | 29 |
| 3.6 | Images of the simulation environment. (a) Describes a tree in the forest.(b) Describes individual trees accumulated to simulate a forest. The simulator has various setting to change camera layout, image registration and spanning humans. Image at bottom describes the image registration image sequence . . . . . | 30 |
| 3.7 | A screenshot of environment with humans at the forest floor. The panel on top right has various attributes that could be customized. . . . .                                                                                                                                                                           | 31 |

|      |                                                                                                                                                                                                                                                                                                                                                  |    |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1  | (a) Relation aperture of lens, depth of field and region of focus. (b) Image showing how for a fixed focus point, the DOF can bring things in and out of focus. . . . .                                                                                                                                                                          | 33 |
| 4.2  | Dynamically Reparameterized Light Field modelling . . . . .                                                                                                                                                                                                                                                                                      | 37 |
| 4.3  | Modeling of system showing one ray in focus while other is not . . . . .                                                                                                                                                                                                                                                                         | 37 |
| 4.4  | With a very large aperture, we can see through objects. No single image in the light field sees the entire hillside. But combining lots of image and selecting the right focal and depth of field creates this result . . . . .                                                                                                                  | 38 |
| 4.5  | A schematic explaining the AOS principle. Multiple photos at aperture $a$ and $h$ above ground level are merged as a 4D light field. Exemplary rays for a synthetic focal plane target point $F$ (cyan) and an out-of-focus occluder $O$ are illustrated (magenta). Dotted rays are partially occluded. Illustration inspired from [2] . . . . . | 40 |
| 4.6  | Left - An image of JKU-ICG dataset[3][4] in LFR visualizer; Right - A synthetic image where white box shows the enhanced features for object detection . . . . .                                                                                                                                                                                 | 41 |
| 5.1  | Montage showing all the images which are part of the VIFB evaluation framework . . . . .                                                                                                                                                                                                                                                         | 44 |
| 5.2  | A flow chart describing the paradigm of transfer learning . . . . .                                                                                                                                                                                                                                                                              | 49 |
| 5.3  | Flow of Region Proposal Networks as single unified detection model . . . .                                                                                                                                                                                                                                                                       | 50 |
| 5.4  | Region Proposal Network Backbone . . . . .                                                                                                                                                                                                                                                                                                       | 50 |
| 5.5  | A mathematical representation of anchor box used in YOLO . . . . .                                                                                                                                                                                                                                                                               | 53 |
| 5.6  | A visual of all the anchor boxes created during prediction . . . . .                                                                                                                                                                                                                                                                             | 53 |
| 5.7  | Comparison of image showing the reduction of bounding box after applying Non-Maximum Suppression . . . . .                                                                                                                                                                                                                                       | 54 |
| 5.8  | A visual representation of the confusion matrix . . . . .                                                                                                                                                                                                                                                                                        | 56 |
| 5.9  | The description of the unified detection by YOLO family of models. Notice how the class probability map are combined with bounding boxes for final detection . . . . .                                                                                                                                                                           | 58 |
| 5.10 | The backbone network of YOLOv3 depicting spatial pyramid pooling . . . .                                                                                                                                                                                                                                                                         | 59 |
| 5.11 | A comparison of all the available networks for YOLOv5 along with there size and inference time . . . . .                                                                                                                                                                                                                                         | 60 |
| 6.1  | Flow chart mentioning all the experiments performed . . . . .                                                                                                                                                                                                                                                                                    | 61 |
| 6.2  | (a) The LiDAR mounted on the tripod. (b) and (c) The API for LiDAR accessible over port on same network. The panel provides interface for configuring scan meta data. . . . .                                                                                                                                                                    | 63 |
| 6.3  | (a) The LiDAR placed at 35m from vegetation. The side and back view objects were concealed behind bushes. (b) The black box encompass the objects partially visible behind leaves. . . . .                                                                                                                                                       | 64 |
| 6.4  | Point cloud of experiment 1, with zoomed region of interest and the overall point cloud on bottom right . . . . .                                                                                                                                                                                                                                | 64 |
| 6.5  | (a) The LiDAR was installed on top of the building nearby the tree (b) The black box encompass the objects partially visible behind leaves. . . . .                                                                                                                                                                                              | 65 |

|      |                                                                                                                                                                                                                                                                                                       |    |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 6.6  | The point cloud visualized in CloudCompare. It could not register anything below the canopy . . . . .                                                                                                                                                                                                 | 66 |
| 6.7  | Process flow of Fusion Framework using MST-SR . . . . .                                                                                                                                                                                                                                               | 67 |
| 6.8  | Result of fusion on the RGB image where few people are hidden inside the shadow of the light source . . . . .                                                                                                                                                                                         | 67 |
| 6.9  | Result of fusion on the greyscale image of a person hidden behind the light source . . . . .                                                                                                                                                                                                          | 68 |
| 6.10 | Result of fusion on the images of a person holding the object behind a non-contrasting background . . . . .                                                                                                                                                                                           | 69 |
| 6.11 | Result of fusion on the images of the object behind the occlusion of vegetation . . . . .                                                                                                                                                                                                             | 70 |
| 6.12 | Bar chart showing the imbalance of classes in FLIR thermal dataset . . . . .                                                                                                                                                                                                                          | 71 |
| 6.13 | Results of detection on the FLIR test data using YOLOv5. The right-most image is the bounding box created using the thermal modality, whereas the middle image is the matching RGB image. The blue box represents bicycles, the red box represents cars, and the green box represents people. . . . . | 72 |
| 6.14 | (a) Top view of the forest. (b) Red dots indicating position of humans . . . . .                                                                                                                                                                                                                      | 73 |
| 6.15 | Python script to perform AOS . . . . .                                                                                                                                                                                                                                                                | 74 |
| 6.16 | Image 1 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.17 | Image 2 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.18 | Image 3 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.19 | Image 4 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.20 | Image 5 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.21 | Image 6 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.22 | Image 7 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.23 | Image 8 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.24 | Image 9 . . . . .                                                                                                                                                                                                                                                                                     | 75 |
| 6.25 | Image 10 . . . . .                                                                                                                                                                                                                                                                                    | 75 |
| 6.26 | Image 11 . . . . .                                                                                                                                                                                                                                                                                    | 75 |
| 6.27 | Image 12 . . . . .                                                                                                                                                                                                                                                                                    | 75 |
| 6.28 | Image 13 . . . . .                                                                                                                                                                                                                                                                                    | 76 |
| 6.29 | Image 14 . . . . .                                                                                                                                                                                                                                                                                    | 76 |
| 6.30 | Image 15 . . . . .                                                                                                                                                                                                                                                                                    | 76 |
| 6.31 | The final integral image from AOS on synthetic data . . . . .                                                                                                                                                                                                                                         | 76 |
| 7.1  | The significant contribution of this work. Red indicates negative experiments, while green indicates positive experiments. . . . .                                                                                                                                                                    | 77 |



# List of Tables

|     |                                                                                                                             |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | A comparative study of various active and passive sensors with their relative cost and foliage penetration ability. . . . . | 22 |
| 3.2 | Essential specifications of FARO Premium 350 Laser Scanner . . . . .                                                        | 25 |
| 5.1 | A summary of fusion methods benchmarked against the metrics and runtime                                                     | 45 |
| 5.2 | Summary of all the benchmarking metrics . . . . .                                                                           | 46 |
| 6.1 | The parameters used to register scans for each experiment. . . . .                                                          | 63 |
| 6.2 | Result of mAP calculation of FLIR Thermal Dataset . . . . .                                                                 | 72 |

# Abbreviations

|            |                             |
|------------|-----------------------------|
| <b>AOS</b> | Airborne Optical Sectioning |
| <b>DOF</b> | Depth Of Field              |
| <b>EDA</b> | Exploratory Data Analysis   |
| <b>IR</b>  | InfraRed                    |
| <b>LFR</b> | Light Field Renderer        |
| <b>RGB</b> | Red Green Blue              |
| <b>SAI</b> | Synthetic Aperture Imaging  |
| <b>SAR</b> | Search And Rescue           |
| <b>UAV</b> | Unmanned Aerial Vehicle     |

*Dedicated to my passion for Robotics...*

# Chapter 1

## Introduction

### 1.1 Motivation

Occlusion of object due to vegetation is a crucial challenge for remote sensing applications in many different domains, including but not limited to search and rescue, detection of wildfires, observation of animals, surveillance, and control of borders, among other domains. Detection of objects under the dense canopy of forests enables numerous advantages to the authority carrying the operation. Hence, technologies developed for these tasks might have disruptive economical impact.

In particular, Search and Rescue (SAR) operations are of utmost importance and need technological advancements. Search and Rescue Operations are carried out with the primary goal of rescuing as many victims as possible who are stranded in an area that has been impacted by a catastrophe. All of these kinds of operations have the same underlying purpose, which is to secure the survival of as many of the afflicted individuals as possible. Hence, more often than not, the speed at which this operation is carried out directly correlates to the success of the operation.

Another important aspect of search operations is towards security aspects. The forest's canopy allows very little light to penetrate and the ground beneath reflects even little.

Hence, most airborne search tasks involve high human inputs and thus bring about huge human, time and monetary resources. Concluding, development of technology which can be deployed on UAVs to aid detection of objects, in particular humans, has the following applications:

1. Quicker responses in disaster invoked search and rescue operations, where autonomous detection might help targeted response and hence more people rescued.
2. To enable development of autonomous aerial vehicles for surveillance in forests - improving security and intruder detection.

## 1.2 The Problem Statement

With the broader aspect in mind, this section formally define the problem statement decided for this thesis. **This thesis seeks to develop a framework to analyse and combine data received from multiple sensor modalities and to present a road map for a pipeline that can assist in identifying objects (primarily humans) concealed by the partial occlusion of forests.** The following flow chart illustrates various broad deliverables by breaking the problem statement into actionable objectives.

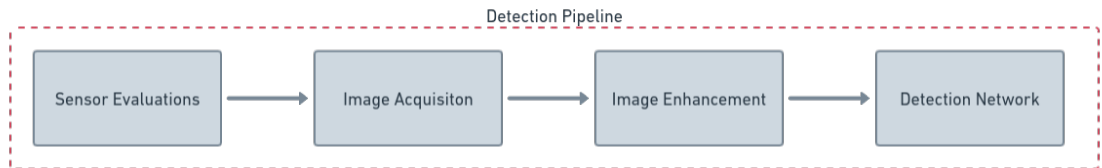


FIGURE 1.1: Flow chart depicting the blocks of pipeline

The fundamental characteristic of the work should be that it makes positive steps toward the establishment of an end-to-end pipeline by establishing actual proof-of-concepts for each of the preceding building components. The anticipated result of the research presented in this thesis is a functional proof-of-concept.

However, optimisation of time and space computational complexity is not one of the key aims of the work presented in this dissertation. The core product, on the other hand, is a functioning baseline framework that could be basis to future iterations. Hence, the work of this thesis would function as a proof-of-concept for further real-time applications.

### 1.3 Outline of Thesis

The thesis begins with establishing the motivation followed by the problem addressed in this work. The second chapter describes a few fundamental principles used throughout the work, which will be of great assistance to readers who are unfamiliar with the background required to understand the work. The chapter builds the context of occlusion, followed by a comprehensive literature survey covering the full history of machine learning systems from their conception to modern CNN and their training. The third chapter included a discussion of numerous tools, such as sensors and simulation environments, as well as a description of the training dataset. The fourth chapter is a detailed chronicle on image acquisition that describes the required mathematical formulation for the reader to comprehend AOS, a crucial aspect of this study. In a similar fashion, the fifth chapter describes the network architecture for object detection and the mathematics underlying the image fusion technique. The sixth chapter is the heart of the study, listing all the experiments with their methods and describing the results and supporting observations. In the final chapter, the work's scope is revisited, and its accomplishments and limitations are evaluated and analysed. In addition, some suggestions for future work to build upon the current work is discussed.

## Chapter 2

# Background

This chapter introduces the reader to the core vocabulary utilised in this thesis, which is especially important for those who are new to the topic. It may also serve as a review of topics for a seasoned reader. Only concepts relevant to comprehending the advancements connected to the basic principles offered in this thesis are addressed.

### 2.1 Establishing Occlusion and its context

Occlusion is a word having multiple contexts depending on the scenario. The most common definition of occlusion, used in literary contexts is “Occlusion is the effect of one object in a 3-D space blocking another object from view.” However in the context of Computer Vision (object tracking problems) and Computer Graphics (ambient occlusion), the word encapsulated various meanings. Below are few scenarios providing examples to set some context of occlusion. Lastly, we describe the context of occlusion pertaining to this work.

1. **Object Tracking** - When building a system that tracks things (people, automobiles, etc.), occlusion happens when one of the items being tracked is concealed (occluded) by another object. As in two people strolling past each other or a vehicle driving

beneath a bridge. The issue here is what to do when an item vanishes and reappears. This type of occlusion is fairly studied in object tracking.

2. **Object Detection** - When building systems for object detection, the dataset must contain ample scenarios of how object might look in the view captured by sensor (eg. camera, laser scanners etc). It might happen that the scene in the image is highly cluttered, and hence the rich features of object might have been hidden behind another object, or partially visible from mesh as shown in Fig. 2.1 (a) To detect objects under such scenario is an active area of research in computer vision, and various deep learning based methods have been deployed for the same.
3. **Sensor Based** - Occlusion is the absence of information in a region when utilising a range camera. Some laser range cameras function by beaming a laser onto the surface you are observing, and then setting up a camera to capture a picture that shows the laser's point of contact. This information provides the point's 3D coordinates. But since the camera and laser are not always in alignment, there may be areas on the surface being inspected that the camera can see but the laser can't reach (occlusion). Here, sensor configuration is more of the issue.

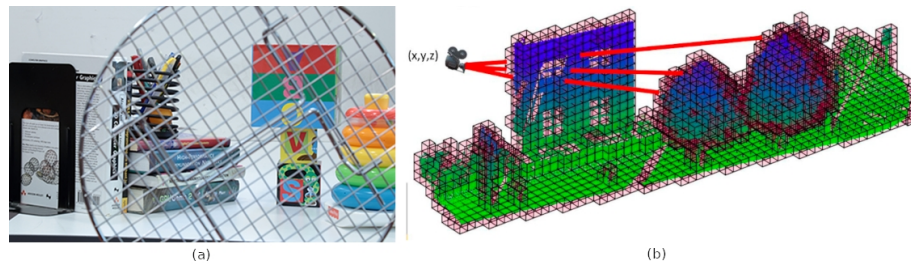


FIGURE 2.1: (a) An example of occlusion due to one object being hidden partially behind other object, in this case the mesh hiding objects behind. (b) An example of occlusion due to sensor configuration.

However, the occlusion in context of this work is due the vegetation cover of leaves in canopy. Fig. 2.2 depicts what kind of perspective we might be dealing with. It is important to note that that image is created by a specialized sensor, called Geiger-mode Lidar[1]. The sensor is proprietary and removes the occlusion of vegetation to show the objects beneath. Occlusion brought on by vegetation is an important challenge for remote



sensing applications in many different fields, including but not limited to search and rescue, detection of wildfires, observation of animals, surveillance, and control of borders, among other fields. This type of occlusion mostly relates to that of Object Detection described above. Since we are primarily interested in detecting objects (humans or animals having heat signatures), we need not remove vegetation entirely. The need is to create images that suppressed features of leaves and samples larger extend of object signatures. Hence, completely removing occlusion is not the primary task, instead sampling more signals of object presence and combining them is focused upon.

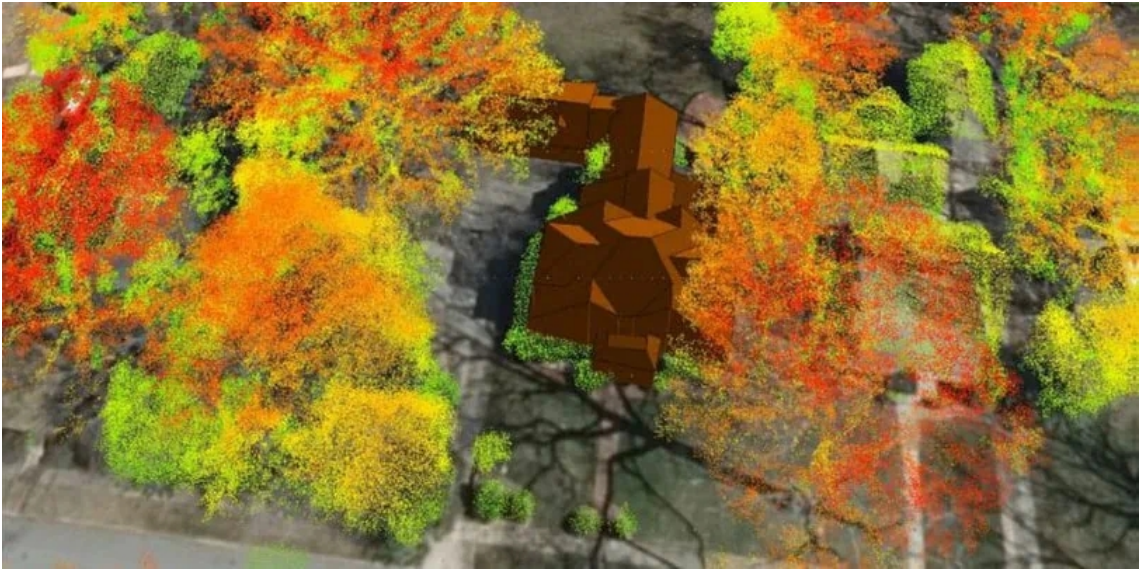


FIGURE 2.2: Building structure captured beneath tree canopy. Image: VeriDaaS, used with permission.[1]

## 2.2 The Problem of Recognizing Objects in Images

The notion of detecting object is varied for different researchers, and hence the problem is vaguely defined. As a result, it is critical to comprehend the many types of particular but related challenges that the term object detection encompasses. Various sorts of object detection challenges may be characterised using the definitions presented in [5] as follows:

1. **Image Classification:** Image classification is the process of classifying and identifying groups of pixels or vectors within an image according to predetermined criteria.

The classification law may be formulated using one or more spectral or textural properties. Various datasets which can be used for image classification tasks include MNIST, CIFAR, and ImageNet. An example of the Image Classification task is given in Fig. 2.3 (a). An image classification network should output man riding a bicycle or cyclist

2. **Object Localization:** Object Localization is the process of determining the location of one or more objects within an image and drawing a bounding box around the extent of those objects. ImageNet dataset with auxiliary bounding box information can be used for performing object localization tasks. 3D object localization is an important area of study which helps in obtaining essential information such as the pose of the object in real world.
3. **Object Recognition:** Object identification challenges include the algorithm locating and classifying all identified items in a given image. In general, the job entails recognizing areas that include an item and then categorizing the discovered object inside that region. PASCAL and MS COCO datasets may be utilized to train neural networks for Object Recognition applications.
4. **Semantic Segmentation:** Semantic segmentation, often known as image segmentation, is the problem of grouping together sections of an image that correspond to the same object class. Because each pixel in a picture is categorized according to a category, this is a kind of pixel-level prediction. Cityscapes, PASCAL VOC, and ADE20K are some examples of benchmarks for this task.
5. **Instance Segmentation:** Instance segmentation is a type of semantic segmentation that focuses on finding instances of items and identifying their borders. As a result, in instance segmentation, semantic segmentation is performed with the labeling of pixels in accordance with the instance of the object to which it belongs.
6. **Keypoint Detection:** Keypoint detection entails simultaneously detecting and localizing the key points of individuals. Key Points and interest points are the same.

They are spatial locations or spots inside the image that determine what is noteworthy or fascinating. They are invariant to picture rotation, shrinkage, translation, distortion, etc., and are, thus, crucial characteristics in the correspondence problem.



FIGURE 2.3: A montage describing the difference the difference of various object recognition techniques.

## 2.3 Neural Networks

After establishing the many types of problem statements that fall within the object recognition paradigm, it is necessary to comprehend the fundamental mathematical theory underlying the tools, i.e. neural networks, utilized to solve these issues. Despite the fact that there are several types of neural networks in A.I. and computer vision, they all share some fundamental notions, which are highlighted in this section. In addition, a basic understanding of convolution arithmetic and two types of convolution operations extensively employed in 2D object identification methods are presented.

### 2.3.1 Perceptrons

A single-layer neural network is known as a perceptron. Stacking Perceptrons results in a multi-layer Perceptron configuration, which is sometimes referred to as a neural network. The perceptron is categorized as a linear classifier having four components. The input values, weights and biases, the weighted sum of inputs, and the activation function make up a Perceptron. Lastly, to produce the necessary output, a bias is introduced to the weighted sum of inputs routed via an activation function.

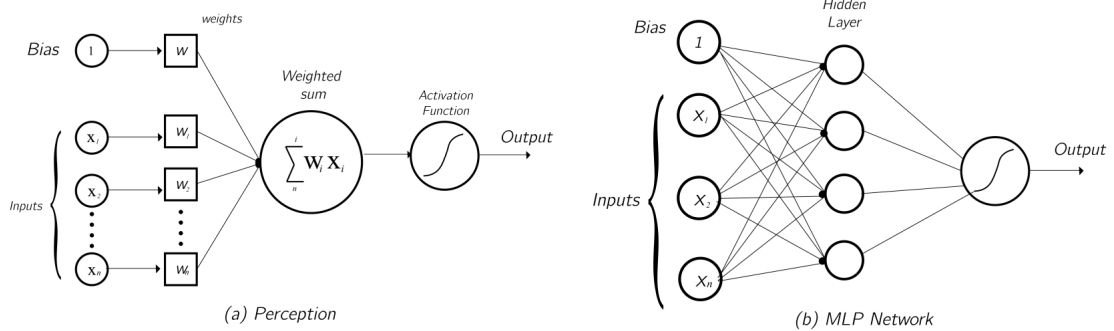


FIGURE 2.4: (a) Pictorial representation of a perceptron. (b) A pictorial representation of how single perceptrons make a MLP network.

A Perceptron is essentially a linear binary classifier, which divides the input hyper-plane into two pieces linearly. As a result, it's clear that a Perceptron could only learn basic

binary functions. To duplicate more sophisticated operations, several perceptrons are layered in layers to form a Multi-Layer Perceptron (MLP) network, also known as a neural network. Hidden layers are the layers between the input and final output layers in these networks. More hidden layers are frequently associated with the network learning increasingly complicated functions, as a general rule. Deep learning networks are based on this concept.

### 2.3.2 Convolution

Convolution (insert complete convolution theory here) The MLP networks are capable of completing regression tasks and eventual classification, but they are unable to deal directly with image data, which is simply a 2D matrix. To extract features from image data, Convolution Neural Networks (CNN) are utilized. Convolution has been a prominent feature in signal processing. In signal processing theory, convolution is defined as an operation over a time-dependent function  $f(t)$  as:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau \quad (2.1)$$

It's essentially an integral product of two functions, one of which has been inverted and moved. The filters are not reversed in machine learning arithmetic, and the inputs are produced via a sliding inner product. Thus, it should be referred to as cross-correlation rather than convolution, but the misunderstanding has become widespread in the research community, so we could continue using the same terminology for the purpose of this work.

The goal of convolution operations is to derive important characteristics from a dataset's input. These qualities aid the network's decision-making. The following are some important terms related to convolution operations, as defined in [6]:

1. Kernel and Filters - A kernel is a two-dimensional array with weights as its elements. A 3D structure consisting of multiple kernels stacked together as channels are referred

to as a filter. Each kernel focuses on a different component of the incoming dataset. The kernel's size correlates to the field of view of the layers of the network

2. Stride - During a convolution process, it is defined as the kernel's step size as it slides over the input matrix.
3. Padding - Padding is used to regulate the size of the output matrix by augmenting the image's boundaries. The size of the output  $o$  is given by the following formula for an input image of size  $i \times i$  operated on by a kernel of size  $k \times k$ , padding  $p$  with stride  $s$ :

$$o = \lfloor \frac{i + 2p - k}{s} \rfloor + 1 \quad (2.2)$$

### 2.3.3 Training a Neural Network

The complete steps in training a neural network (or any ANN) are the following -

1. Configure data set
2. Set network architecture
3. Train neural network.people
4. Improve generalization performance (fine tuning)
5. Test results
6. Deploy model

We will only focus on After understanding the fundamental structure and mathematical theory underlying MLP networks and convolution operations, a generic procedure for the training protocol can be defined. The training of a network, also known as back-propagation [7], is separated into four stages:

1. Forward Pass: Random values are used to initialize the kernels, and the final output gets determined using the input data.

2. **Loss Computation:** The eventual aim of the detection procedure is to minimize loss mathematically expressed as loss function. Using the true target vector, the loss value is determined based on the output produced, by computing various loss functions.
3. **Backward Pass:** This technique involved calculating the rate of change in loss with changes in weight values to determine which weights contributed the most to the loss. Essentially, gradients of the loss function with respect to each weight are computed to determine the rate of change of loss wrt weight. Mathematically, it is described as:

$$\frac{\partial L}{\partial W} \quad (2.3)$$

- (a) **Weight Update:** The kernel's weights are updated using the equation below, and the cycle is continued until the loss is minimized:

$$w_{k+1} = w_k - \lambda \frac{\partial L}{\partial w_k} \quad (2.4)$$

- (b) The learning rate  $\lambda$  is a hyperparameter that determines how quickly the minima is reached.

### 2.3.4 Parallel between Control Theory and Neural Network training

In a control system, a setpoint is the value at which it is designed to operate, and is often called the goal value. After a setpoint (input) has been specified, it is processed by a controller, which then modifies the value of the setpoint in accordance with the feedback loop (Manipulated Variable). After the setpoint has been modified, the data is sent to the system that is being controlled, where it is used to generate an output. This output is monitored using the proper metric, and then, with a feedback loop, the metric mathematically compared to the initial input. This enables the controller to specify the degree of modification (amount of adjustment) that will be made to the initial setpoint using the “Manipulated Variable.”

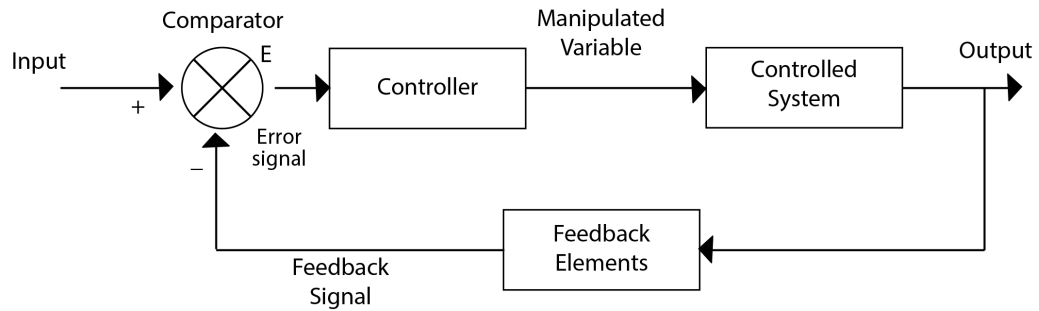


FIGURE 2.5: A traditional control system

Take for instance the resistance (controlled system) found in a radiator as an example. Imagine you decide to keep the temperature in the room at 20° degrees Celsius (set-point). The controller instructs the radiator to begin operating, at which point it feeds the resistance with a level of power that it has already determined. After that, a probe (thermometer) will take the temperature of the surrounding environment (feedback elements), which will then be compared (comparator) to the setpoint (desired temperature), which will then cause the electric intensity that is supplied to the resistance to be adjusted (controller). An incremental step for adjustment is used in the process of deploying the new intensity.

The process of training a neural network is analogous to the operation of a example in the sense that the regulated system is the detection model for objects (say cats and dogs). The aim is now to minimise the Loss (error function) between the classification of the input data and the one determined by the neural network. The goal differs in this case since the goal is no longer to have the smallest difference between the setpoint temperature and the actual temperature.

In order for this to be accomplished, the system will need to take into consideration the input (analogous to setpoint), and calculate an output, which is the controlled system, using the parameters that are established by the algorithm. The term "forward pass" refers to this particular stage.



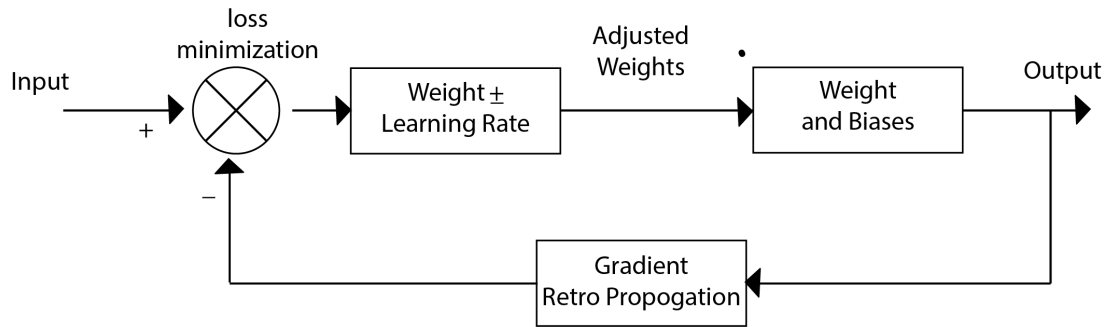


FIGURE 2.6: A diagram drawing comparisons between control system and neural network

After the output has been determined, the system will use a technique called Gradient Retro-propagation to re-propagate the evaluation mistake (Feedback Elements or Back-propagation). While the difference in temperature measured by the thermometer and the setpoint was turned into electrical intensity for the radiator, now the system would modify the weights of the various inputs into each neuron based on a particular step size (learning rate).

### 2.3.5 Convolution Neural Networks

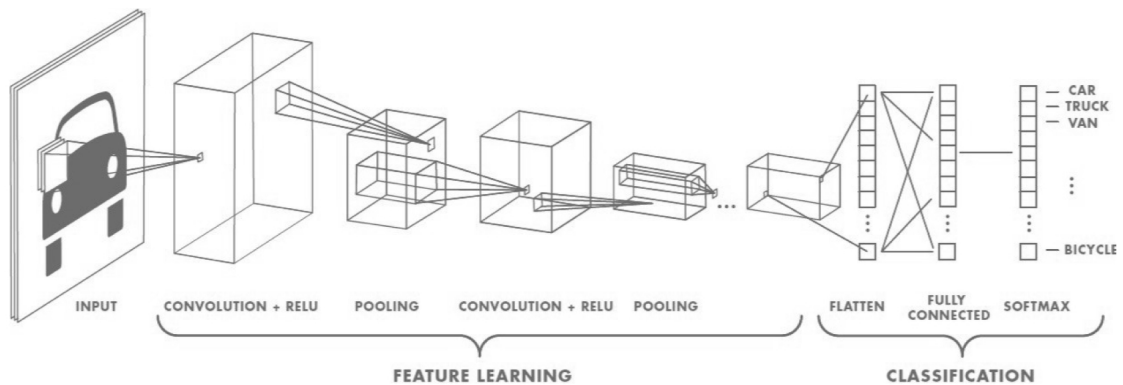


FIGURE 2.7: Overview of CNN learning features and finally classifying for the given task

A Convolutional Neural Network, also known as a CNN, is a type of deep learning system that can take an input image, assign significance (in the form of learnable weights and biases), and differentiate between the numerous characteristics and objects that are

contained within that image. The amount of pre-processing that is required by a ConvNet is significantly lower in comparison to that required by other classification methods. ConvNets (CNN), on the other hand, may learn these filters and properties with sufficient training, in contrast to basic techniques which require hand-engineering of filters.

ConvNets are preferred over Multi-layer perceptrons because through the application of suitable filters, a ConvNet may effectively capture the Spatial and Temporal relationships in an image. Due to the reduced number of parameters involved and the reusability of weights, the architecture performs superior fitting to the image dataset. In other words, the network may be taught to better grasp the image's complexity. While in the case of MLP, we will have to flatten a matrix which when used for performing class prediction on relatively simple binary images, may produce an average precision score, but when it comes to complicated images with pixel dependencies throughout, the approach will have little to no accuracy.

A ConvNet will have the following components:

1. Convolution Layer - The convolution layer performs the convolution operation with suitable padding and kernel. The convolution operation's goal is to extract high-level characteristics from the input image, such as edges. ConvNets don't have to have just one convolutional layer. The first ConvLayer is traditionally responsible for capturing low-level characteristics such as edges, color, gradient direction, and so on. With more layers, the architecture adjusts to high-level characteristics as well, resulting in a network that understands all of the images in the dataset in the same way that a human brain would.
2. Pooling Layer - The pooling layer, like the convolutional layer, is responsible for shrinking the spatial size of the convolved feature. Through dimensionality reduction, the computer power needed to process the data is reduced. It's also beneficial for extracting rotational and positional invariant dominating features, which helps keep the model's training process running smoothly. There are two types of pooling operations:

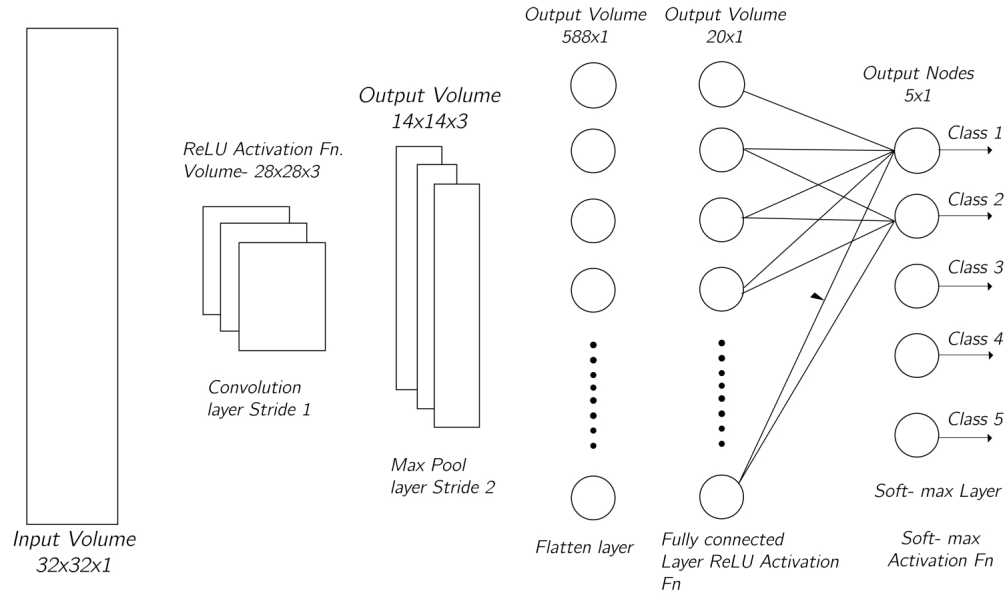


FIGURE 2.8: Depiction of various layers in a CNN with reduction of output vector

- (a) **Average Pooling** - Average Pooling returns the average of all values from the kernel-covered portion of the image. Furthermore, average pooling is only a noise-suppressing strategy that reduces dimensionality.
  - (b) **Max Pooling** - The maximum value from the portion of the image covered by the kernel is returned by Max Pooling. Max Pooling can also be used to reduce noise. It removes all noisy activations and conducts de-noising and dimensionality reduction at the same time. As a result, we may conclude that Max Pooling outperforms Average Pooling.
3. **Classification (Fully Connected Layer)** - Feed forward neural networks are what the Fully Connected Layer is all about. The network's last levels are fully linked layers. The output from the final Pooling or Convolutional Layer, which is flattened and then fed into the fully connected layer, is the input to the fully connected layer. After passing through the fully connected layers, the final layer employs the softmax activation function (rather than ReLU) to determine the likelihood that the input belongs to a certain class (classification). As a result, the completely linked layers aid in the learning of non-linearities that may be necessary for prediction.

### 2.3.6 Loss Functions and Activation Functions

Most machine learning techniques are designed to maximize or decrease a function known as the Loss function. Loss functions are a statistical assessment of how accurate a given model's predictions are. Several aspects, including the existence of outliers, the choice of the algorithm, the time efficiency of gradient descent, the simplicity of discovering the derivatives, and the confidence of predictions, influence the selection of an appropriate loss function [23]. Loss functions may be largely classified into two categories: classification and regression. We would discuss a few loss functions below which are relevant to this work.

#### 2.3.6.1 Classification Loss

Loss functions for classification are meant to operate on discrete labels. These loss functions are useful for object classification tasks, as their name implies. The most commonly used loss function is described below

1. Cross-Entropy Loss - The cross-entropy loss function is the most used classification loss function. Due to its construction, it is also termed as the negative log-likelihood function.

$$L(y^p, y) = -(y_i^p \log(y_i) + (1 - y_i^p) \log(1 - y_i)) \quad (2.5)$$

Therefore, if the true label  $y_i$  is 1, the second portion of the function disappears, and if it is 0, the first portion disappears. Consequently, the cross-entropy loss function consists mostly of the logarithm of the anticipated category label.

#### 2.3.6.2 Regression Loss

These loss functions are used when one wants to predict a continuous real variable using a neural network. We would discuss two fairly common regression loss functions.

1. Mean Squared Error (MSE) - Mean Squared Error is the sum of the square of the difference between the predicted values and the ground truth.

$$MSE = \frac{\sum_{i=1}^n (y_i - y_i^p)^2}{n} \quad (2.6)$$

2. Mean Absolute Error (MAE) - Mean Absolute Error, is the sum of the absolute deviations between predictions and the ground target value. Consequently, it represents the average size of the errors. Whenever the direction of the mistake is also taken into account, it is referred to as the Mean Bias Error (MBE).

$$MAE = \frac{\sum_{i=1}^n |y_i - y_i^p|}{n} \quad (2.7)$$

MAE loss is advantageous for data consisting of outliers. However, the greatest concern is that the gradient remains consistent throughout. Thus, when the discrepancy between predictions and ground truth is small, the algorithm may overestimate and overlook the true value.

### 2.3.6.3 Activation Functions

Activation Functions - In a neural network, an activation function describes how the weighted sum of the input is turned into an output from nodes in a network layer. As a result, it's simply a single function that you utilize to retrieve node output. It is sometimes referred to as the Transfer Function. It is used to evaluate the output of a neural network for tasks such as classification. It converts the resultant values from 0 to 1 or -1 to 1, and so on (depending upon the function). Activation Functions are classified into two types:

1. Linear Activation Function
2. Non-Linear Activation Function

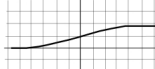
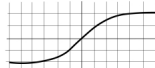
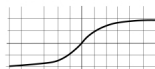
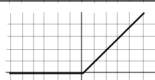
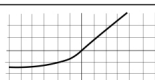
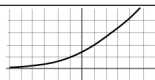
| Name                                     | Plot                                                                                | Equation                                                                                           | Derivative                                                                                     |
|------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Identity                                 |    | $f(x) = x$                                                                                         | $f'(x) = 1$                                                                                    |
| Binary                                   |    | $f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x > 0 \end{cases}$                  | $f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$          |
| Logistic (a. k. a Soft step)             |    | $f(x) = \frac{1}{1 + e^{-x}}$                                                                      | $f'(x) = f(x) (1 - f(x))$                                                                      |
| TanH                                     |    | $f(x) = \tanh(x) = \frac{1}{1 + e^{-2x}}$                                                          | $f'(x) = 1 - f(x)^2$                                                                           |
| ArcTan                                   |    | $f(x) = \tan^{-1}(x) =$                                                                            | $f'(x) = \frac{1}{x^2 + 1}$                                                                    |
| Rectified Linear Unit (ReLU)             |    | $f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$               | $f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$          |
| Parametric Rectified Linear Unit (PReLU) |    | $f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$        | $f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$     |
| Exponential Linear Unit (ELU)            |   | $f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ | $f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x > 0 \end{cases}$ |
| SoftPlus                                 |  | $f(x) = \log_e(1 + e^x)$                                                                           | $f'(x) = \frac{1}{1 + e^{-x}}$                                                                 |

FIGURE 2.9: Tabular representation of various activation function with their graphs, mathematical representation and derivatives.

## Chapter 3

# Description of Sensors, Dataset and Simulation Environment

This chapter will discuss the sensors utilized, as well as explain their operation and data gathering notations. In addition, we will detail the dataset used to train the object detection model and do some basic exploratory data analysis. Finally, we will go through the simulation environment that was used to evaluate the performance of Airborne Optical Sectioning.

### 3.1 Active and Passive Sensors

“The method of detecting and monitoring the physical features of an area from a distance via the use of measurements of the region’s reflected and emitted radiation is known as remote sensing”. The sensors used in remote sensing are broadly categorized as Active and Passive sensors.

1. An active sensor is a sensor that requires an external power source to operate, in context of remote sensing, an active sensor is equipped with a sensor that transmits that sends out a signal that is received by a transmitter. The signal could be a

soundwave, electromagnetic wave or a electron beam. Few examples of active sensors are LiDAR, GPS, RADAR etc.

2. Passive sensors, on the other hand, are forms of remote sensing technology that can detect energy that is readily available in its natural state. In order for passive sensors to detect energy, there must first be naturally occurring energy present in the environment. When the sun or some other light source is lighting the Earth, and only then, is it possible for any and all reflected energy to occur. RGB Cameras, Thermal Cameras, and other such devices are examples of passive sensors.

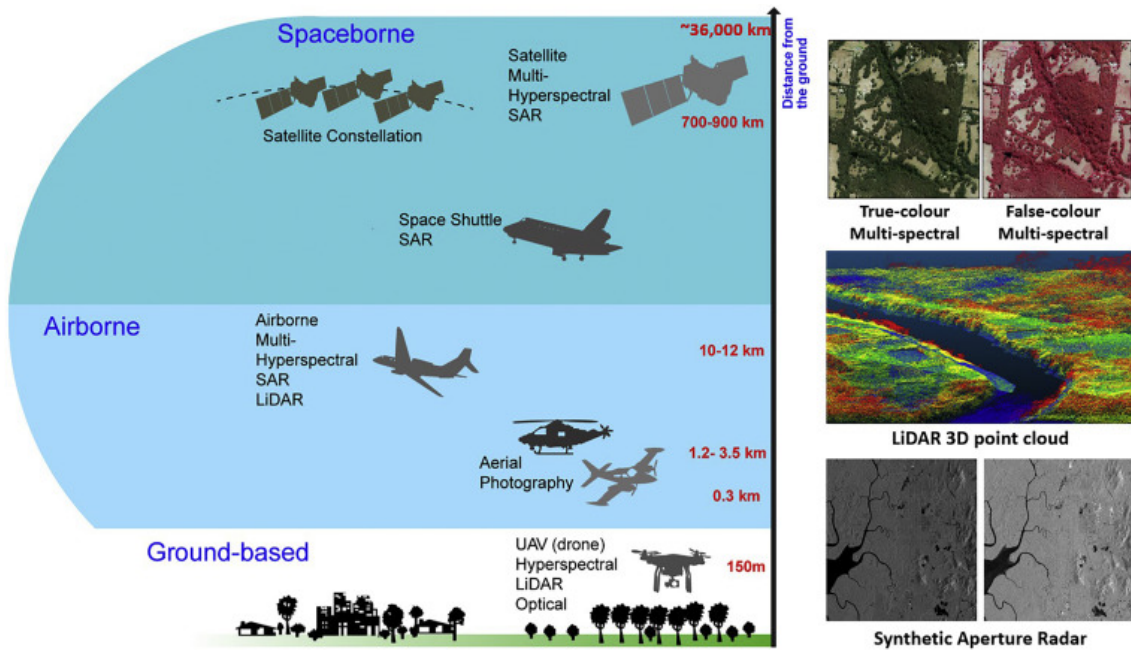


FIGURE 3.1: Image shows the range of various sensors used in remote sensing and along with the vehicle they are mounted on. This work primarily entails to ground-based region ( $<150\text{m}$ ) and hence LiDAR and Optical Sensors are shortlisted keeping in mind their compatibility with UAV.

Since detecting human/animal presence beneath the partial occlusion of leaves (of canopy) correlates most to remote sensing a study of sensors commonly used in remote sensing was done. Below is a table summarizing a few sensors used for remote sensing with their ability to sample signals of human presence beneath a partial occlusion of leaves.



|                                                  | RGB Camera                                   | Thermal Camera                              |
|--------------------------------------------------|----------------------------------------------|---------------------------------------------|
| Active/Passive                                   | Passive                                      | Passive                                     |
| Cost of sensor                                   | Low                                          | Medium                                      |
| Ability to map foliage cover                     | High                                         | Medium                                      |
| Ease of deployment on drones                     | High                                         | High                                        |
| Ability to register humans beneath foliage cover | Medium when sparse cover, low in dense cover | High in sparse cover, medium in dense cover |
|                                                  | SAR                                          | Hyperspectral Camera                        |
| Active/Passive                                   | Active                                       | Passive                                     |
| Cost of sensor                                   | High                                         | High                                        |
| Ability to map foliage cover                     | High                                         | Low                                         |
| Ease of deployment on drones                     | Low                                          | Medium                                      |
| Ability to register humans beneath foliage cover | High                                         | Rarely used, data unavailable               |
|                                                  | LiDAR (Airborne)                             | Multispectral Camera                        |
| Active/Passive                                   | Active                                       | Passive                                     |
| Cost of sensor                                   | Medium to High                               | High                                        |
| Ability to map foliage cover                     | Depends on point cloud sampling density      | Low                                         |
| Ease of deployment on drones                     | Medium                                       | Medium                                      |
| Ability to register humans beneath foliage cover | Low, difficult to distinguish points         | Rarely used, data unavailable               |

TABLE 3.1: A comparative study of various active and passive sensors with their relative cost and foliage penetration ability.

## 3.2 RGB and Thermal Camera

Thermal cameras are passive sensors that detect infrared radiation generated by any objects with temperatures greater than absolute zero. The use of this sort of sensor in vision systems overcomes the lighting issues that plague traditional grayscale and RGB cameras. The camera of the smartphone CAT S60 was used to capture the image for the proof of concept test.



FIGURE 3.2: The CAT S60 smartphone with locations of IR and RGB sensors

The primary camera has 13 MP, AF, and an uncooled FLIR infrared camera (Lepton module). Using an embedded FLIR app, the phone can do thermal imaging, heat palettes,

and temperature spot measurements. It has many image capture modes, however Black Hot and White Hot are utilized for assessment. In both of these modes, heat signals from humans can be picked up behind trees due to temperature difference. The camera also supports RGB image acquisition, which was utilized to capture RGB pictures of the relevant scenes. The poses for the images are calculated using the Pinhole camera model, by using the open source library COLMAP. Below is brief summary of the Pinhole camera model.

### 3.2.1 Pinhole Camera Modeling

Using the pinhole camera model, a 3D scene can be converted to a 2D scene. It encompasses the relationship between the 3D world coordinates and the 2D image coordinates using projective geometry. It is to be noted that the model assumes the camera aperture to be a pinhole and does not model the use of lens to focus the image onto the image plane. Hence, it does not model the physical phenomenon such as lens distortion and is good as a first order approximation.

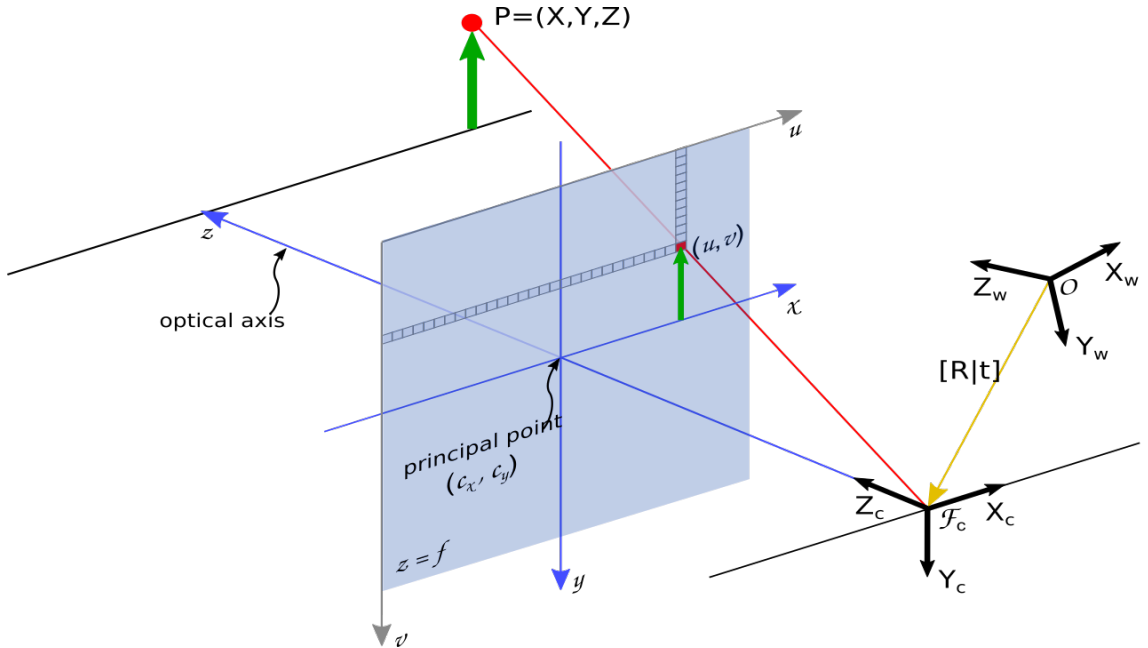


FIGURE 3.3: A pictorial representation of pinhole camera model

The pinhole camera model provides the mathematical formulation that projects a scene from 3D points to 2D image coordinates formulating a perspective transformation.

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (3.1)$$

where:

- $(X, Y, Z)$  represent coordinates of a 3D point in world frame.
- $(u, v)$  are the coordinates (in pixels) of the projection of  $(X, Y, Z)$  on the image plane.
- $K$  represents intrinsic camera parameters.
- $[R \mid t]$  are extrinsic camera parameters, mapping world space to camera space. It is composed of a 3D rotation followed by translation.
- $(c_x, c_y)$  are camera's principal point (in pixels).
- $f_x, f_y$  represent the camera's horizontal and vertical focal lengths, respectively, in pixel units.
- $s$  is the scaling factor.

Since the cameras used in this work is a cellphone cameras which are already calibrated, the images are free from distortion and hence pose information derived from those are not prone to distortion errors.

### 3.3 Laser Scanner - FARO Focus Premium 350

3D Laser Scanners are instruments used to create 3D laser scans. They are active sensors, and use non-contact, non-destructive technique that uses a stream of laser light to digitally

record the geometry of actual items. While used for various tasks, the one used in this work was primarily meant for creating high resolution surface terrains.

The FARO Focus 350 is a single mode return LiDAR which uses laser triangulation to create a state vector of points in real world to create a 3D point cloud. The LiDAR operates in 2 modes, one capturing the RGB information hence each point in the cloud is 6 dimensional tuple -  $(X, Y, Z, r, g, b)$ . Here  $(X, Y, Z)$  are the world coordinates with the origin at sensors position as shown in Fig. The LiDAR also has another mode of operation which captures the intensity of reflection of white and black. In this case a point in point cloud is a 4 dimensional tuple  $(X, Y, Z, I)$ , with  $I$  being the intensity value. The LiDAR is heavy and needs to be mounted on a tripod for stationary scans. The scans time vary according to the resolution.

| Specifications (FARO Premium 350) |                                                     |
|-----------------------------------|-----------------------------------------------------|
| Power Supply                      | 19 V (external supply), 14.4 V (internal battery)   |
| Typical Power Consumption         | 19 W idle, 32 W scanning, 72 W charging             |
| Typical Battery Operation Time    | About 4 hours                                       |
| Typical Scan Time                 | ~1-5 Mins                                           |
| Weight                            | 4.4 Kgs (including battery)                         |
| Range                             | 153m at 2MPts/sec                                   |
| 3D Accuracy                       | 2mm @ 10m, 3.5mm @ 25m                              |
| Ranging Error                     | ~1mm                                                |
| Field of View                     | 300° vertical / 360° horizontal                     |
| Step Size                         | 0.009° (40,960 Pts on 360°) vertical and horizontal |
| Laser Class and Wavelength        | Laser Class 1 with frequency 1553.5 nm              |

TABLE 3.2: Essential specifications of FARO Premium 350 Laser Scanner

### 3.4 Training Dataset

The primary objective of this work was to develop a pipeline to evaluate the performance state-of-the-art object detection models while trying to detect object beneath the partial occlusion of forest canopy. Hence, transfer learning was used to train a YOLOv5 model. Two datasets were primarily used for training and testing purposes - COCO (Common Objects in Context) and FLIR Thermal Dataset. The overview of the datasets is presented below.

### 3.4.1 Overview of COCO Dataset

The COCO dataset [8] stands for Common Object in Context dataset and is widely used by the research community. The dataset also has its own annotation format known by the same name. What makes this dataset unique from others is that it addressed three core research problems in scene understanding -

1. Providing non-iconic views with iconic views of objects
2. The contextual reasoning between objects
3. Precise 2D localization of objects

From the official definition “COCO is a large-scale object detection, segmentation, and captioning dataset”. The highlight features of COCO dataset are:

1. Object segmentation
2. Recognition in context
3. Superpixel stuff segmentation
4. 330K Images with over 200K labeled
5. 80 object categories (over 1.5 million object instances)

Out of the multiple COCO datasets, but the one relevant to this work was the one with object detection annotations. For an object detection task, the model should create the bounding boxes around discrete objects while simultaneously assigning them one of the 80 categories. The file format used by the COCO dataset is JSON, which has a dictionary (key-value pairs inside curly braces {...}) as the top value. The dictionary can have nested dictionaries or an ordered collection of items (lists inside square braces [...]).

FiftyOne is a tool for downloading, visualizing and evaluating the COCO dataset. The documentation provides ample details about official integration with COCO dataset. A few samples of dataset visualized in FiftyOne is shown in Fig. 3.4

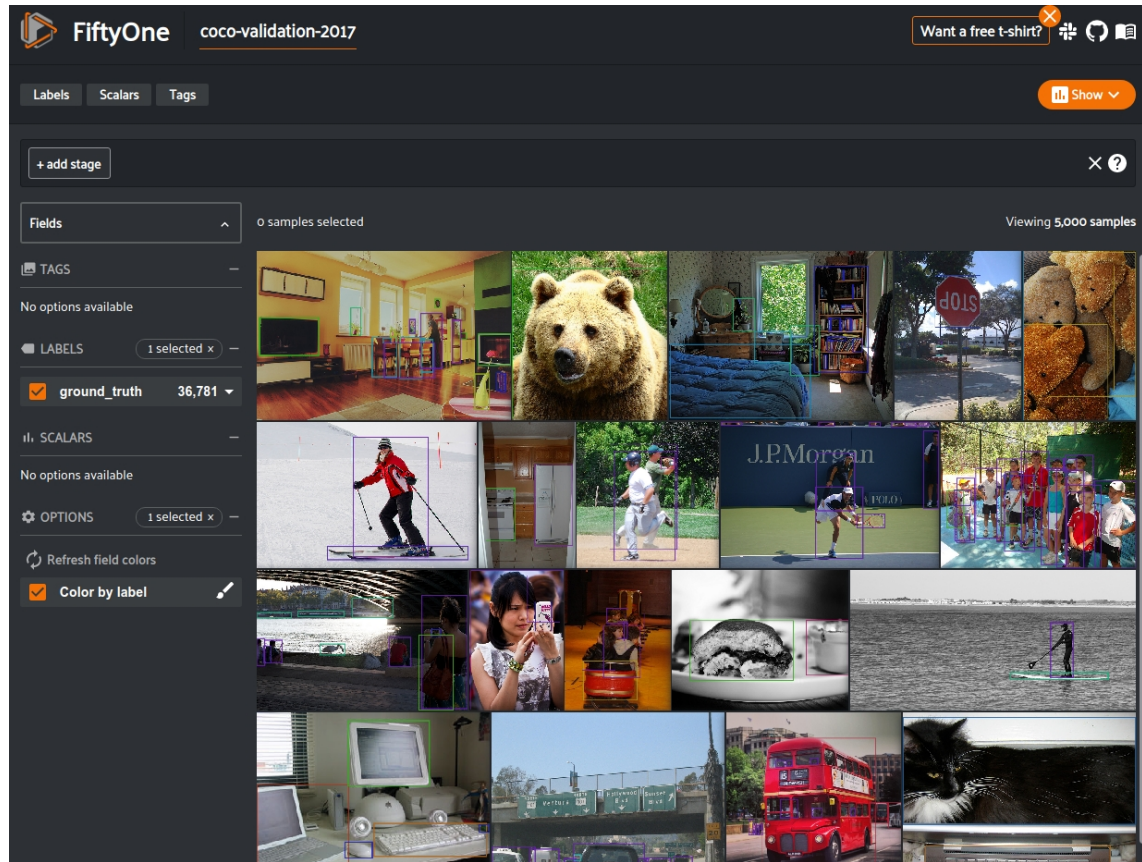


FIGURE 3.4: Some sample images from COCO dataset visualizes in the interface of FiftyOne

The description of various keys in JSON format are -

1. Info - Information pertaining to the description of your dataset as well as its version history.
2. Licenses - Licenses is a list that have their own unique IDs and may be provided by the images uploaded.
3. Categories - Categories are a kind of classification, and each category has its own unique ID. These categories might be connected, but not necessarily required, with any higher-level category that encompasses numerous classes.

4. Images - A list of the images included in your dataset, together with any pertinent information, such as the image's unique ID, its filepath, its height and width, and other optional features, such as a licence, URL, or the date it was taken, etc.
5. Annotations - A list of annotations, each of which has its own unique ID and the image ID to which it is related. This is where the information about the bounding box in our scenario, as well as the information regarding segmentation, keypoints, and other labels for other activities is stored. This also saves the bounding box area and the iscrowd value, both of which indicate that the evaluation will utilise a large bounding box that encompasses several items that fall under the same category.

### 3.4.2 Overview of Teledyne FLIR Thermal Dataset

This free Teledyne FLIR ADAS Dataset[9] contains fully annotated thermal and visible spectrum frames, which may be used for the development of object identification systems using CNNs. This data was generated to enable the automotive community to develop ADAS and autonomous vehicle systems that are safer and more efficient, as well as to spur research on visual and thermal sensor fusion algorithms (also known as "RGBT").

The specifications of the dataset are the following:

- Content - A total of 26,442 frames that have had all of their annotations completed, with 520,000 bounding box annotations spread over 15 distinct object categories.
- Images - A proposed training/validation split, 9,711 thermal and 9,233 RGB training/validation images. The dataset also includes pre-AGC 16-bit frames.
- 7,498 video frames in total were captured at 24 Hz. Thermal and visual frames are identical with a 1:1 match between them. Again the dataset includes pre-AGC 16-bit frames.
- Frame Annotation Label Totals - 3,75,000+ annotations combined in thermal and visible spectrum.

- Video Annotation Label Totals - 1,45,00+ annotations combined in thermal and visible spectrum.
- Thermal Camera Specifications - Teledyne FLIR Tau 2 640x512, 13mm f/1.0 (HFOV 45°, VFOV 37°)
- Visible Camera Specifications - Teledyne FLIR Blackfly S BFS-U3-51S5C (IMX250) camera and a 52.8° HFOV Edmund Optics lens
- Dataset File Format - Thermal 8-bit JPEG (AGC applied), RGB 8-bit JPEG
- Annotations - MSCOCO formatted annotations (JSON)



FIGURE 3.5: Some frames from the Teledyne FLIR ADAS dataset for thermal algorithmic training

### 3.5 The Simulation Environment

To create areal image data that resembles drone video shot during flights over actual forests, a forest simulation was created. The key benefit of this method is that several samples may be evaluated quickly and compared to ground truth data while avoiding the time and investment in real world scans. This can be quickly used to tune the algorithms before real world deployment.

The simulation is based on WebGL, with a technique known as ProcTree2's being used to implement the forest simulation. It computes 512x512 pixel aerial images for drone flights that take place over a preset region and for given sampling parameters (such as the location of the drone, its height, the sample distance, and the field of view of the camera).



The perspective projection is used by the virtual rendering camera, and the look-at vector of the camera is oriented such that it is perpendicular to the ground surface normal (that is, it is pointing downwards). The simulated images were binarized in order to differentiate between occluders above the ground (0, black) and unoccluded points on the ground (1, white).

A random seed generator a seeded random number generator was used to construct trees with varied densities and degrees of resemblance. Using tree characteristics such as tree height (ranging between 20m-25m), trunk length (ranging between 4m-8m), trunk radius (ranging between 20cm-50cm), and leaf size (ranging between 5cm-20cm), a combination of conifer and broadleaf trees were generated. These tree leaves were taken since they had textures available open sourced. The simulation is based on this work.[10][11]



FIGURE 3.6: Images of the simulation environment. (a) Describes a tree in the forest.(b) Describes individual trees accumulated to simulate a forest. The simulator has various setting to change camera layout, image registration and spanning humans. Image at bottom describes the image registration image sequence

The simulator also has options to spawn humans, which can be given color and texture for benefited identification. The humans can do various tasks such as sitting, walking, standing, laying on ground etc. For the purpose of this study, the humans are all standing with skin color texture.



FIGURE 3.7: A screenshot of environment with humans at the forest floor. The panel on top right has various attributes that could be customized.

## Chapter 4

# Image Acquisition - Synthetic Aperture Imaging and Airborne Optical Sectioning

In this chapter, we will extend the basics of the pinhole camera model to establish a relationship between the aperture of the sensor (camera in our case), the depth of field, and its use to highlight the object in focus. We will then establish the basics of Light Field Rendering, relate it to Dynamically Reparameterized Light Fields and lastly describe AOS and its application to increase the synthetic aperture for occlusion removal.

### 4.1 Camera aperture and its relation to Depth of Field

Camera aperture or commonly referred to as lens aperture is the opening of the lens which controls the amount of light that will enter the lens onto the image sensor. Therefore, a large aperture lets more light in and hence produces a shallow depth of field while a small aperture leads to a large depth of field. The physics behind the depth of field is related to the part of the lens that is being used while forming an image. This is a field of study in physics but a qualitative relation can be established as follows - The depth of

field shows the range of distances across which the image will be roughly in focus. The light cone narrows when the aperture is reduced. This suggests that the confusion circles are becoming smaller. As a result, the range of distances at which the picture is in focus has expanded.

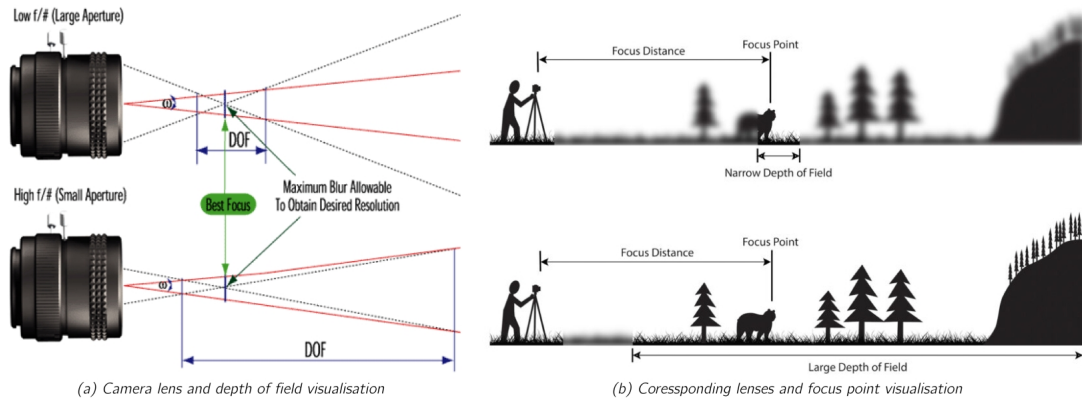


FIGURE 4.1: (a) Relation aperture of lens, depth of field and region of focus. (b) Image showing how for a fixed focus point, the DOF can bring things in and out of focus.

**F-number:** The f-number of an optical system, such as a camera lens, is the ratio of the focal length to the diameter of the entry pupil in optics (“clear aperture”). The f-number is often represented by a lower-case hooked  $f$  in the pattern  $f/N$ , where  $N$  is the f-number. The reciprocal of the relative aperture is the f-number (the aperture diameter divided by focal length).

## 4.2 Image based Rendering

Rendering is the process of converting a 3-D scene into a 2-D rendered scene, mostly displayed as an image. This can be most simply achieved with a camera. Though, in classical computer graphics, more often than not 2-D images are used to generate 3-D scenes which are used to create 3-D models to render novel scenes from different perspectives. Geometry is the foundation of traditional computer graphics approaches. In other words, they create photographs of 3-D scenes based on geometric representations of such situations. The intricacy of these approaches is determined on the geometry of the sceneries. Furthermore,

the accuracy with which these approaches can portray the interactions between light and a surface is restricted. This is because conventional methodologies rely on lighting and surface models that are often an approximation of reality. Geometry-based modeling's usefulness is limited by these two constraints, especially in real-time applications.

Image-based rendering is an alternative option. Rendering an image of a scene using this method entails determining the value of an appropriate subset of the plenoptic function for that scene. The benefit of this method is that if the plenoptic function is represented relatively well, it is feasible to create fresh perspectives of a scene without ever having to worry about geometry, lighting, or surface models. As a result, the cost of interactively seeing the scene is independent of scene complexity. As a result, image-based rendering enables the utilization of several two-dimensional images to instantly produce unique two-dimensional visuals, bypassing the manual modeling step. Though the thesis does not directly use the principles of image based rendering, the idea of mathematical representation of rays and its association with dynamically reparameterized light field and depth of field is essential for realizing synthetic aperture imaging.

### 4.3 Light Field Rendering

To describe a single beam of light, we must take into consideration the following and quantize them mathematically

1. Spectral association to determine the color
2. The pose in space
3. The direction from which it originated and is traveling
4. The association of its changed with respect to time

Therefore, a single, monochromatic beam of light may be described in detail using seven quantities: three for location  $(x, y, z)$ , two for direction  $(\theta)$  and  $(\phi)$ , one for wavelength

( $\lambda$ ), and one for time ( $t$ ). This is known as the plenoptic function, which is represented as  $P(\theta, \phi, \lambda, t, x, y, z)$  and represents the intensity of light beams as a function of these seven parameters.

Now, this function with 7 variables captures every single light ray in the universe. However, we do not always need to capture all the 7 variables. This is because seeing the complete spectrum of light across all space, in all directions, and throughout all time is unrealistic. For most practical purposes, especially those pertaining to computer vision or graphics, most of the variables are fixed, and hence the image can be represented by a subset of plenoptic function.

Consider the following to demonstrate this concept: A color picture measures the plenoptic function for a particular place, time, and direction throughout the range of frequencies of visible light. Otherwise, this image is the function  $P(\theta, \phi, \lambda, t, x, y, z)$  with  $t, x, y, z$  fixed. Because just three of the parameters ( $\theta, \phi, \lambda$ ) may change, the color snapshot can be thought of as a 3D slice of the plenoptic function. This led to the two most famous representations of Light Field, the 4D and the 5D representations of the light field. For the purpose of this work, we shall describe the 4D light field. [12]

### 4.3.1 4D Light Field

The 5D light field was modeled around the human visual system, with two cameras being modeled as human eyes. Further, human eyes can only perceive RGB spectrum hence dropping the  $\lambda$ . We assume a static image, and hence the time ( $t$ ) can be modified to  $t'$ . The 5D light field representation encompasses enough information to allow completely free rotation of camera, which in-turn introduces redundancy. As a result, we may reduce this 5-D representation to a 4-D subset without incurring considerable loss. This is the subset presented in the paper Light Field Rendering by Levoy and Hanrahan's.

Thus the advantage we have in computer vision is that there can be an infinite number of cameras seeing the same scene. Hence, In the context of computer graphics, we may imagine an endless number of cameras spread out on an x-y plane and pointing in an

infinite number of different directions. This array of cameras would take a measurement at a single point in time to determine the four-dimensional slice of the plenoptic function provided by

The parameter  $z'$  is never used and is a dummy variable since the cameras exist on the  $x$ - $y$  plane and in every direction. Hence, the subset of the plenoptic function displayed in a light field is this 4-D slice of the plenoptic function. This representation includes enough information about the light in a scene for a rendering engine to produce 2-D pictures of the scene from different camera locations. Thus, in their paper, the authors propose a new technique that allows much more freedom in range of possible views while reducing dimensions. As they quote “The major idea behind the technique is a representation of the light field, the radiance as a function of position and direction, in regions of space free of occluders (free space). In free space, the light field is a 4-D, not a 5-D function.” By applying the restriction that light beams in a scene must be of constant value along their propagation routes. The dimensions can be lowered by one by doing so. We intuitively assume that a beam of light is constant throughout its route of travel. In other words, light rays should be represented as the same value throughout the path taken, therefore there is no need to represent them at every location along the path. As a result, preserving the information from one point on the line is enough to describe the complete line. Hence, they parameterize the light to be represented by using just a point and the direction of travel, assuming it was traveling in a space free of occlusion. They parameterize light rays represented as lines by their intersections with two planes in arbitrary positions in the free space. Quoting the paper, “By convention, the coordinate system on the first plane is  $(u, v)$  and on the second plane is  $(s, t)$ . An oriented line is defined by connecting a point on the  $u$ - $v$  plane to a point on the  $s$ - $t$  plane.”

Since we needed two points on two planes each, we would need two local coordinates on both these planes. The light rays can then be represented by a function  $L(u, v, s, t)$ , and hence the function resulting from this two plane parameterization is a 4-D light field. This idea enabled a quick way for image-based rendering of new views, and then in the paper Dynamically Reparameterized Light Fields it was extended to create variable aperture.

### 4.3.2 Dynamically Reparameterized Light Field

The light field introduced earlier was only capable of storing scenes with approximately constant depth at low to moderate sampling, while increasing sampling lead to the task being computationally expensive. Using dynamic reparameterization, one can render images with variable focus and depth of field without the knowledge of scene geometry.

The way this method differs from Light Field Rendering is that instead of using two planes, they model the light raw using a Focal Plane ( $F$ ) and a camera surface  $C$ . This is identical to the pinhole camera model, only with a lot of camera arrays to sample a wide field of aperture. In the previous parameterization, each ray  $r$  is uniquely determined by a tuple  $(s, t, u, v)$ . In contrast to that, this parameterization is based on a camera surface  $C$ , parameterized using the coordinates  $(s, t)$  which is identical to the 4D method. Now, each camera is represented as  $D_{s,t}$  and is unique, though in the context of this work they are identical in internal calibration. Now, each pixel on the camera surface is represented with coordinates  $(u, v)$ . Hence, a ray  $r$  is represented as  $(s, t, u, v)$ . Now, since the focal surface is dynamical in nature, each ray can intersect it at  $(f, g)$ . Due to its dynamic nature, we subscript the coordinates  $(f, g)_F$ . Thus we use an alternative naming  $(s, t, f, g)_F$ . The theory is described in detail in the original manuscript [13], while here we would discuss the application for variable aperture which is relevant to this work.

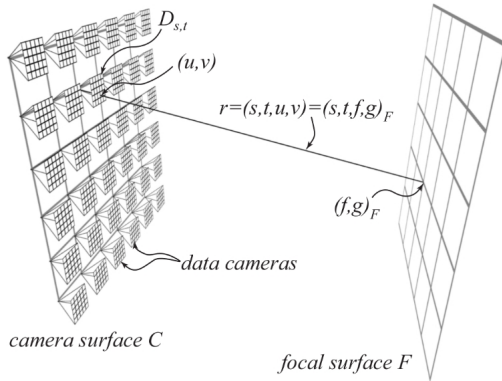


FIGURE 4.2: Dynamically Reparameterized Light Field modelling

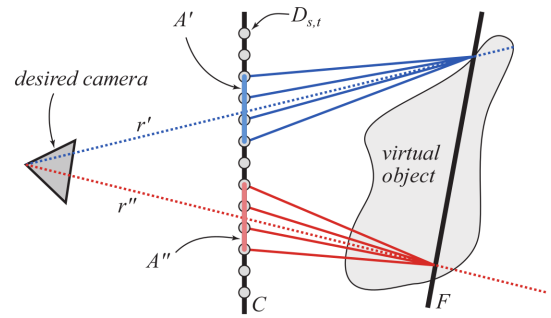


FIGURE 4.3: Modeling of system showing one ray in focus while other is not



### 4.3.3 Variable Aperture

In Fig. 4.3 , we are attempting to reconstruct two rays, which are denoted by the letters  $r'$  and  $r''$ . Within the context of this illustration, the breadth of our synthetic apertures  $A'$  and  $A''$  is comprised of four data cameras. The intersection of  $r'$  and  $r''$  with the camera surface  $C$  serves as the focal point for the synthetic apertures that we create. After that, we get ray database samples by applying  $\mathbf{M}_{s,t}^{F \rightarrow D}$  for all  $(s, t)$  in such a way that  $D_{s,t}$  is contained inside the aperture. These  $s$  and  $t$  data are merged such that a single reconstructed ray may be produced. It is important to note that  $r'$  does not cross  $F$  at the surface of the virtual object, despite the fact that  $r''$  does intersect  $F$ . Because of our reconstruction using a synthetic aperture,  $r'$  will be shown as being in focus, while  $r''$  will not. The amount of depth-of-field may be changed by adjusting the size of the synthetic aperture.

This method has the capability of generating synthetic apertures of any size. The maximum size of the aperture can never exceed the number of cameras that are mounted on the surface of the camera. Given that we would be utilising the same camera in a variety of geotagged locations, this is mostly limited by the amount of computational accuracy that can be achieved. When our apertures are wide enough, we are able to look partially through objects, as seen in Fig. 4.4

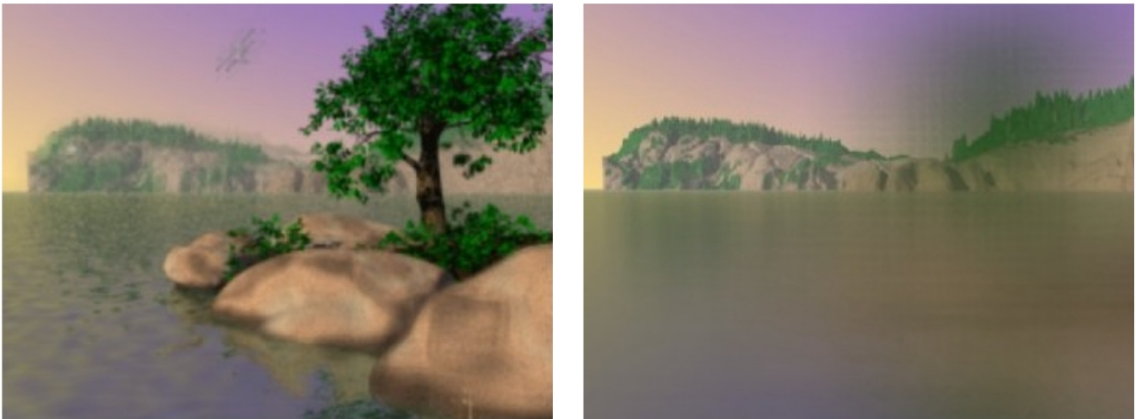


FIGURE 4.4: With a very large aperture, we can see through objects. No single image in the light field sees the entire hillside. But combining lots of image and selecting the right focal and depth of field creates this result

## 4.4 Airborne Optical Sectioning

### 4.4.1 Synthetic Aperture Imaging

Synthetic aperture is a technique to approximate the signal of a single hypothetical wide aperture sensor from an array of multiple small aperture sensors or single sensors at multiple positions. The data from all the sensors are computationally combined and integrated to augment the depth-of-field, resolution etc. This idea has been widely used among the industry, such as in ultrasound, sonar, optical imaging etc. There are various kinds of synthetic aperture techniques such as SAR, SART, SAS, SAU, SAL, ISAM, SLFI and AOS. The bulk of the methods that have been discussed so far are active, meaning that they send out electromagnetic or sound waves while simultaneously monitoring the backscattered signal. On the other hand, SART, SLIF, and AOS are all passive and can only receive signals.

### 4.4.2 Airborne Optical Sectioning (AOS)

AOS [14] is a synthetic aperture imaging technique which is based on the Dynamically Reparameterized Light Field modeling of a scene. It consists of two basic steps -

1. Wide Synthetic Aperture Sampling
2. Wide Synthetic Aperture Visualization

Now, one can either use an array of cameras for registering multiple images, or use drones with accurate localization to sample a series of images. If a drone is used, we can fix the FOV of the drone camera and use it as a basis to calculate the necessary height and sampling area and sampling resolution. This is described in detail in the work Airborne Optical Sectioning, though for the purpose of this work, a light field renderer is created and these parameters are tuned manually using a slider for proof-of-concept purposes.

The advantage AOS over other techniques is-

1. It does not need an expansive sensor setup and works entirely on image-based rendering. Hence this method is ideal for use with a drone
2. The method does not calculate 3D point clouds and does not depend on correspondence problems such as the case with LiDAR or Stereo Cameras.
3. Large samples can lead to extremely large apertures, and hence the extremely shallow depth of field. If the occluders are sufficiently sparse, this blurs the occluders and points out the ground target.

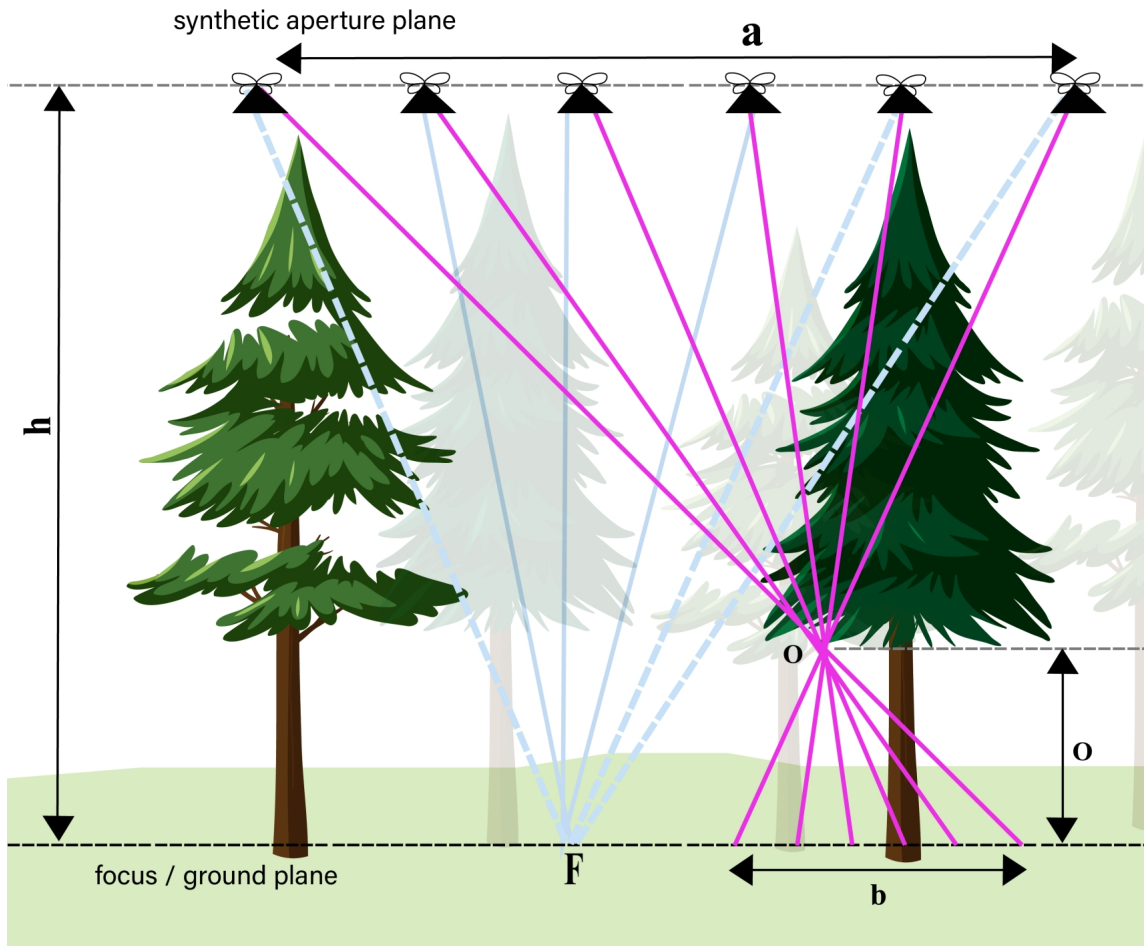


FIGURE 4.5: A schematic explaining the AOS principle. Multiple photos at aperture  $a$  and  $h$  above ground level are merged as a 4D light field. Exemplary rays for a synthetic focal plane target point  $F$  (cyan) and an out-of-focus occluder  $O$  are illustrated (magenta). Dotted rays are partially occluded. Illustration inspired from [2]

### 4.4.3 Light Field Renderer Module

To visualize AOS, an application interface was created with options to tweak parameters. The application was compiled in C++ with python binding to improve ease of use. The code was inspired by this work. [15]

Fig. 4.6 shows a screenshot of the visualization tool for Light Field Rendering algorithm. It is the backbone of AOS, and requires essentially four things - synthetic aperture (or Field of View (FoV)), pose of images, images and the focus planes. The visualization tool has option to load images, read poses and then assign the position, FoV and near and far plane. The option for pinhole renders image basis the pinhole camera model. The open function runs the LFR and visualizes the synthetically generated image. As seen in Fig. 4.6, the synthetic image has suppressed the features not in plane while the blobs imitating humans are still visible (portion marked by white rectangles).

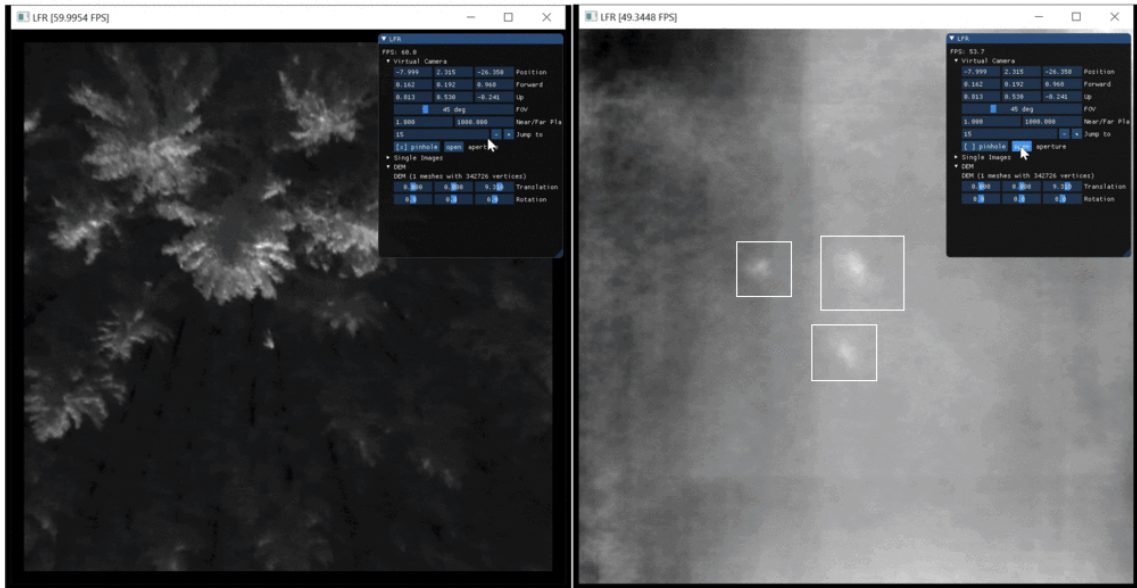


FIGURE 4.6: Left - An image of JKU-ICG dataset[3][4] in LFR visualizer; Right - A synthetic image where white box shows the enhanced features for object detection

## Chapter 5

# Image Fusion and Object Detection

In this chapter, we will discuss and formulate the framework for fusing the RGB and Thermal (IR) images. We will then describe the Transfer Learning paradigm, YOLOv5 architecture, and describe the training process.

### 5.1 Visible and Thermal Image Fusion

The purpose of image fusion is to build a single image that contains information extracted from many images in order to produce an image that is both more informative and more easily processed in the future. Mostly the image fusion algorithms fuse the images by dividing them into either pixel-level, feature-level, or the decision-level. Image fusion technology is broadly classified into numerous forms based on application fields, viz-a-viz medical image fusion, multi-focus image fusion, remote sensing image fusion, and visible and infrared image fusion.

This work primarily involved using visible and thermal (infrared) image fusion algorithms. Before the advent of deep learning, image fusion algorithms were generally categorized

as multi-scale transforms, saliency and subspace-based, sparse representation based, and amalgamation of multiple techniques (hybrid). It is clear that deep-learning-based methods can provide better features as compared to handcrafted features, and provide hybrid weights which are crucial for learning fusion in well-studied settings. Though, it can be safely said that blindly applying deep-learning based methods without sufficient data would lead to erroneous results. Hence, a common benchmarking dataset and evaluation metrics are essential to evaluate and choose the most suitable fusion method for the task.

Hence, VIFB: A Visible and Infrared Fusion Benchmark was used to evaluate the techniques by analyzing the output of algorithms on a few sample images of a setting similar to our proof of concept. VIFB has been discussed ahead, with a brief description of evaluation metrics.

### 5.1.1 VIFB

Visible and Infrared image fusion benchmark (VIFB) was a study on recent fusion algorithms. The library has 20 fusion algorithms, with 21 image pairs sourced from the internet encompassing various real world situations. These 21 images were fused and evaluated using 13 evaluation metrics, each useful for different settings the algorithm might be deployed in. The algorithms and evaluation metrics have been tabulated below, while the ones used in this work have been described in detail.

### 5.1.2 Dataset

The dataset of VIFB is a test set, which is acquired by the authors from the Internet and from fusion tracking databases. These image pairings include a broad variety of settings and working situations, such as indoor, outdoor, low light, and over-exposure. This assists in testing the generalization ability of image fusion algorithms.



FIGURE 5.1: Montage showing all the images which are part of the VIFB evaluation framework

### 5.1.3 Fusion Algorithms

VIFB provides methods for 20 fusion algorithms published in the last 5 years. The work also categorizes them into broadly deep learning based algorithms, multi-scale fusion, saliency based, hybrid and other. These algorithms were created to fuse grayscale images, but have been suitably modified to fuse RGB as well. The table list all the methods, their date of publication and category.

### 5.1.4 Benchmarking Metrics

There have been several assessment criteria developed for visible-infrared image fusion. Image fusion metrics, as defined in [16], are grouped into four types: information theory-based, image feature-based, image structure similarity-based, and human perception-based



| Method     | Year | Category       | Average Run Time | Qcb   | Qcv    |
|------------|------|----------------|------------------|-------|--------|
| ADF        | 2016 | Multi-scale    | 1.00             | 0.474 | 777.8  |
| CBF        | 2015 | Multi-scale    | 22.97            | 0.526 | 1575.1 |
| CNN        | 2018 | DL-based       | 31.76            | 0.622 | 512.6  |
| DLF        | 2018 | DL-based       | 18.62            | 0.445 | 759.8  |
| FPDE       | 2017 | Subspace-based | 2.72             | 0.46  | 780.1  |
| GFCE       | 2016 | Multi-scale    | 2.13             | 0.535 | 898.9  |
| GFF        | 2013 | Multi-scale    | 0.41             | 0.619 | 881.6  |
| GTF        | 2016 | Other          | 6.27             | 0.414 | 2138.4 |
| HMSD-GF    | 2016 | Multi-scale    | 2.76             | 0.604 | 533    |
| Hybrid_MSD | 2016 | Multi-scale    | 9.04             | 0.623 | 510.9  |
| IFEVIP     | 2017 | Other          | 0.17             | 0.462 | 573.8  |
| LatLRR     | 2018 | Saliency-based | 271.04           | 0.497 | 697.3  |
| MGFF       | 2019 | Multi-scale    | 1.08             | 0.542 | 676.9  |
| MST_SR     | 2015 | Hybrid         | 0.76             | 0.645 | 522.7  |
| MSVD       | 2011 | Multi-scale    | 1.06             | 0.426 | 809    |
| NSCT_SR    | 2015 | Hybrid         | 94.65            | 0.617 | 1447.3 |
| ResNet     | 2019 | DL-based       | 4.80             | 0.445 | 724.8  |
| RP_SR      | 2015 | Hybrid         | 0.86             | 0.606 | 888.8  |
| TIF        | 2016 | Saliency-based | 0.13             | 0.545 | 613    |
| VSMWLS     | 2017 | Hybrid         | 3.51             | 0.497 | 754.7  |

TABLE 5.1: A summary of fusion methods benchmarked against the metrics and runtime

metrics. However, none of the offered measures outperforms the others. Since this work resembles most human perception inspired based tasks, the algorithm collectively performing well in those metrics was chosen for fusion.

## 5.2 Multi-scale Transform and Spare Representation Fusion

Multiscale transform (MST) and sparse representation (SR) [17] are widely used concepts in signal processing theory. To keep things simple, just the fusion of two source photos is addressed, although the suggested framework may easily be expanded to fuse more than two images. The comprehensive fusion strategy includes the four processes listed below.

1. MST Decomposition - The first step involves performing the specific MST on two source of images (denoted as  $I_A$  and  $I_B$ ) to process their low-pass bands (denoted as



| Category                    | Name  | Meaning                             |
|-----------------------------|-------|-------------------------------------|
| Information theory-based    | CE    | Cross entropy                       |
|                             | EN    | Entropy                             |
|                             | MI    | Mutual information                  |
|                             | PSNR  | Peak signal-to-noise ration         |
| Structural similarity-based | SSIM  | Structural similarity index measure |
|                             | RMSE  | Root mean squared error             |
| Image feature-based         | AG    | Average gradient                    |
|                             | El    | Edge intensity                      |
|                             | SD    | Standard deviation                  |
|                             | SF    | Spatial frequency                   |
|                             | QAB/F | Gradient-based fusion performance   |
| Human perception inspired   | QCB   | Chen-Blum metric                    |
|                             | Qcv   | Chen-Varshney metric                |

TABLE 5.2: Summary of all the benchmarking metrics

$L_A$  and  $L_B$ ) and the corresponding high-pass bands (for uniformity denoted as  $H_A$  and  $H_B$ )

## 2. Low-pass fusion -

- (i) Using the sliding window approach, split  $L_A$  and  $L_B$  into image patches of size  $\sqrt{n} \times \sqrt{n}$  using a step of  $s$  pixels from higher left to lower right. Assume  $T$  patches are indicated by  $\{p_A^i\}_{i=1}^T$  and  $\{p_B^i\}_{i=1}^T$  and in  $L_A$  and  $L_B$ , respectively.
- (ii) Following which each position indexed with  $i$  is rearranged by modifying  $\{p_A^i, p_B^i\}$  into column vectors as  $\{\mathbf{v}_A^i, \mathbf{v}_B^i\}$ . They are then normalized by modifying each vector's mean value to zero to obtain  $\{\hat{\mathbf{v}}_A^i, \hat{\mathbf{v}}_B^i\}$  by

$$\begin{aligned}\hat{\mathbf{v}}_A^i &= \mathbf{v}_A^i - \bar{v}_A^i \cdot \mathbf{1}, \\ \hat{\mathbf{v}}_B^i &= \mathbf{v}_B^i - \bar{v}_B^i \cdot \mathbf{1},\end{aligned}$$

where  $\mathbf{1}$  denotes an all-one valued  $n \times 1$  vector,  $\bar{v}_A^i$  and  $\bar{v}_B^i$  are the mean values of all the elements in  $\mathbf{v}_A^i$  and  $\mathbf{v}_B^i$ , respectively.

- (iii) The next step involves calculating the sparse coefficient vectors  $\{\alpha_A^i, \alpha_B^i\}$  of

$\{\hat{\mathbf{v}}_A^i, \hat{\mathbf{v}}_B^i\}$  using the OMP (orthogonal matching pursuit) algorithm by

$$\begin{aligned}\alpha_A^i &= \arg \min_{\alpha} \|\alpha\|_0 \quad \text{s.t.} \quad \|\hat{\mathbf{v}}_A^i - \mathbf{D}\alpha\|_2 < \varepsilon, \\ \alpha_B^i &= \arg \min_{\alpha} \|\alpha\|_0 \quad \text{s.t.} \quad \|\hat{\mathbf{v}}_B^i - \mathbf{D}\alpha\|_2 < \varepsilon,\end{aligned}$$

here  $\mathbf{D}$  denotes the learned dictionary.

(iv) This step involves obtaining the fused sparse vector by merging  $\alpha_A^i$  and  $\alpha_B^i$  with the “max-L1” rule

$$\alpha_F^i = \begin{cases} \alpha_A^i & \text{if } \|\alpha_A^i\|_1 > \|\alpha_B^i\|_1 \\ \alpha_B^i & \text{otherwise} \end{cases}$$

The fused result of  $\mathbf{v}_A^i$  and  $\mathbf{v}_B^i$  is denoted by

$$\mathbf{v}_F^i = \mathbf{D}\alpha_F^i + \bar{v}_F^i \cdot \mathbf{1},$$

where the merged mean value  $\bar{v}_F^i$  is given by

$$\bar{v}_F^i = \begin{cases} \bar{v}_A^i & \text{if } \alpha_F^i = \alpha_A^i \\ \bar{v}_B^i & \text{otherwise} \end{cases}.$$

(v) Iterate the above process for all the source image patches in  $\{p_A^i\}_{i=1}^T$  and  $\{p_B^i\}_{i=1}^T$  to obtain all the fused vectors  $\{\mathbf{v}_F^i\}_{i=1}^T$ . Let  $L_F$  denotes the low-pass fused result. For each  $\mathbf{v}_F^i$ , reshape it into a patch  $p_F^i$  and then plug  $p_F^i$  into its original position in  $L_F$ . As patches are overlapped, each pixel’s value in  $L_F$  is averaged over its accumulation times.

3. High-pass fusion -  $H_F$  is obtained by combining  $H_A$  and  $H_B$  using the “max-absolute” technique, with the absolute value of each coefficient serving as the activity level measurement. Then, utilise the consistency verification technique to ensure that a fused coefficient did not come from a different source image than the majority of its neighbours. The small majority filter must be used to accomplish and verify this.
4. MST reconstruction - Corresponding inverse MST is performed over  $L_F$  and  $H_F$  to reconstruct the final fused image  $I_F$ .

## 5.3 Evaluation Metrics

The mathematical representation of the benchmarking metrics used in this work has been described in this section.

### 5.3.1 Chen-Blum Metric

Chen-blum metric (CBM) [18] primarily compares the contrast features to the features of source images with that of a fusion image. It is mathematically defined by

$$Q_{CB} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N \lambda_{ij}^A c_{ij}^{AF} + \lambda_{ij}^B c_{ij}^{BF} \quad (5.1)$$

where  $\lambda^X$  is the saliency map and  $c^{XF}$  is the contrast preserved in the fusion image from source image  $X$ .

### 5.3.2 Chen-Varshney Metric

Chen-Varshney Metric  $Q_{CV}$  [19]. This metric is inspired by 'human perception' and compares the 'edge information' between source and fused images. It is calculated as follows:

$$Q_{CV} = \frac{\sum_{l=1}^L \left( \lambda \left( I_A^{W_l} \right) D \left( I_A^{W_l}, I_F^{W_l} \right) + \lambda \left( I_B^{W_l} \right) D \left( I_B^{W_l}, I_F^{W_l} \right) \right)}{\sum_{l=1}^L \left( \lambda \left( I_A^{W_l} \right) + \lambda \left( I_B^{W_l} \right) \right)} \quad (5.2)$$

where  $D \left( I_A^{W_l} \right)$  and  $D \left( I_B^{W_l} \right)$  is the similarity measure in the local region  $W$ ,  $\lambda(.)$  represents the local region saliency value,  $W_l$  is a local region,  $L$  denotes the number of nonoverlapping regions, the detailed description can be found in [56].

## 5.4 Object Detection Network

This section describes the state-of-the-art object detection framework YOLO, the transfer learning paradigm and PyTorch implementation of YOLOv5. Essential topics for understanding YOLO such as non-maximal suppression and anchor boxes has also been described.

### 5.4.1 Transfer Learning Paradigm

In recent years, as the complexity of neural networks has increased, training a network from start has become an increasingly challenging and time-consuming endeavour. Because of this, an innovative idea known as Transfer Learning [18] has emerged as a viable alternative to traditional training pipelines. Transfer learning refers to the process of using the information that a model has obtained while being trained for a different job using a different dataset in order to accomplish the present goal. The main concept behind transfer learning is to maximise efficiency. Transfer learning is not technically a technique of machine learning; rather, it is more of a design paradigm that is actively used in a variety of computer vision applications. There

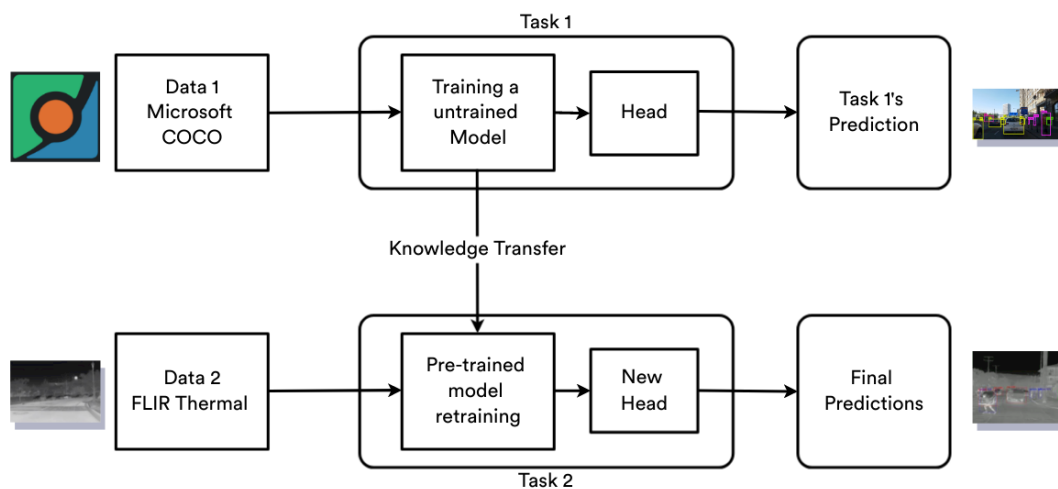


FIGURE 5.2: A flow chart describing the paradigm of transfer learning

are two common approaches to Transfer Learning paradigm. They are the develop model approach and the pre-trained model approach. We have used pre-trained model approach which works as follows:

- (a) **Select Source Model:** From among the available models, a pre-trained source model is selected for use. There are many research institutes that publish models on extensive and difficult datasets, and these models might be added to the pool of models to choose one.
- (b) **Reuse Model:** After the model has been pre-trained, it may be utilised as the basis for another model that will perform the second task which is of interest. Depending on the modelling approach that was used, this might require using the whole of the model or only a portion of it.
- (c) **Fine-tune Model:** There is also the possibility that the model will need to be modified or improved based on the input-output pair data that is readily accessible for the activity of interest.

### 5.4.2 Region Proposal Networks

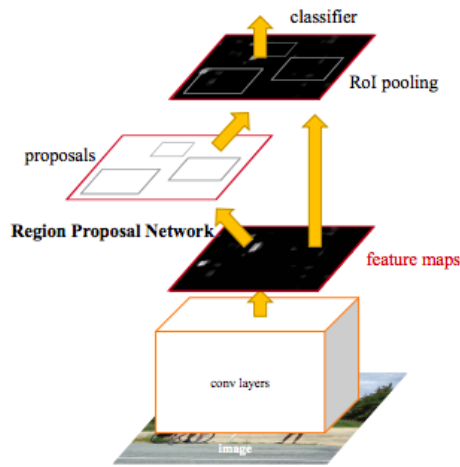


FIGURE 5.3: Flow of Region Proposal Networks as single unified detection model

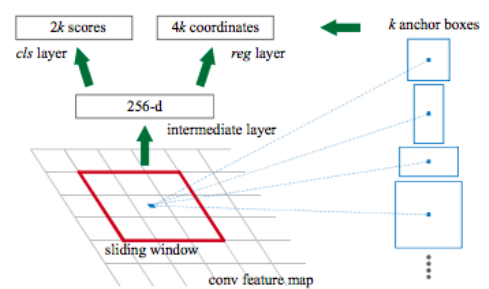


FIGURE 5.4: Region Proposal Network Backbone

Predicting areas of interest is yet another important notion that is used extensively by contemporary CNN networks for the purpose of object localisation. In order to

assess whether or not certain parts of an input image contain objects, an object detection network would often sample a large number of these regions. After that, it regresses these areas to determine the bounding box based on the fact that an object is present in the region where the regressing is taking place. Region Proposal Networks (RPN), which were made famous by the Faster-RCNN [20] network, are used by the vast majority of the object detectors that are widely used today.

### 5.4.3 Anchor Boxes

YOLO networks popularised the usage of Anchor boxes, hence enabling YOLO to improve its performance drastically over the performance of current RPN networks. YOLO can predict successfully even when numerous objects are linked with a single grid cell. In the scenario of overlap, however, when a single grid cell includes the centres of two distinct objects, anchor boxes are employed to enable a single grid cell to identify several objects.

Anchor boxes, also known as antecedent boxes, are precursor areas that serve as customised templates for a certain class of image objects to which the network maps other regions of interest. The shape and size of the anchor boxes are based on

- (a) The class of object they are designed for i.e. if the objects are small, the anchor boxes are small. This is encoded in the parameter size ( $s$ ),  $s \in (0,1]$
- (b) The aspect ratio of the bounding boxes that are labelled in for various object classes in training dataset. This is denoted by  $r(>1)$ .
- (c) The dimensions of input image,  $w \times h$ , where  $w$  is the width and  $h$  is the height.

Hence, the size of anchor box becomes  $ws\sqrt{r} \times hs/\sqrt{r}$ . The network generally creates lots of anchor boxes, and the steps followed by RPN for matching anchor boxes during training include:

1. Following the generation of a large number of anchor boxes, the network labels each one according to the position and category of the target object that is

included inside it. This process is analogous to the ground truth bounding box data that is made accessible from the training dataset.

2. Assuming the previous step generated  $n_a$  anchor boxes given as  $\{A_1, A_2, \dots, A_{n_a}\}$  for an image with  $n_b$  ground truth bounding boxes  $\{B_1, B_2, \dots, B_{n_b}\}$ , s.t.  $n_a \geq n_b$ , the algorithm then defines a matrix  $X \in \mathbb{R}^{n_a \times n_b}$ , where each distinct element  $x_{ij}$  stores the IoU between  $A_i$  anchor box and  $B_j$  bounding box.
3. The greatest element in the matrix  $X$ , or the prediction with the highest IoU, is identified by its index  $(i_1, j_1)$ , and the matching bounding box  $B_{j_1}$  is given to the anchor box  $A_{i_1}$ . The remaining items in matrix  $X$ 's  $i_1^{th}$  row and  $j_1^{th}$  column are all ignored.
4. The procedure is continued using the matrix  $X$  subsequent-highest member up until all of the elements in its  $n_b$  columns have been eliminated. This arrangement is equivalent to the case when each bounding box is allocated to a different anchor box.
5. If the IoU between two anchor boxes is larger than a particular specified threshold, which is often set at 0.5, then the two anchor boxes are allocated to a bounding box. This process is repeated for the remaining  $n_a - n_b$  anchor boxes. These anchor boxes are referred to as Positive anchor boxes, and they are each given a category name that is comparable to the bounding box to which they have been allocated. After that, the offset target vector is computed for each positive anchor box by subtracting the centres and diameters of the bounding boxes from those of the anchor boxes:

$$\Delta = \left( \frac{\frac{x_b - x_a}{w_a} - \mu_x}{\sigma_x}, \frac{\log\left(\frac{w_b}{w_a}\right) - \mu_w}{\sigma_w} \right) \quad (5.3)$$

6. The model still needs to distinguish between foreground and background. The other category of any unassigned anchor boxes helps in this, since they are changed to background, and the offset vector is not generated for those anchor

boxes. The RPN network utilises these anchor boxes as instances of negative anchor boxes across the network.

It is worth noting that during the prediction phase, the network creates multiple anchor boxes (as shown in Fig. 5.6) and the above procedure of predicting the categories of object and calculating offsets for each of them is done basis the feature maps. The final prediction result is calculated using a technique called Non-Maximum Suppression so that avoid multiple bounding box creation.

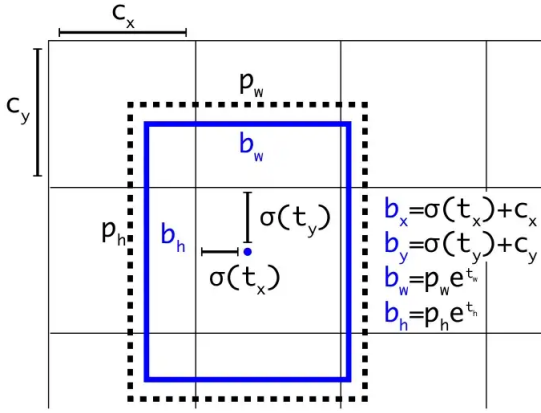


FIGURE 5.5: A mathematical representation of anchor box used in YOLO

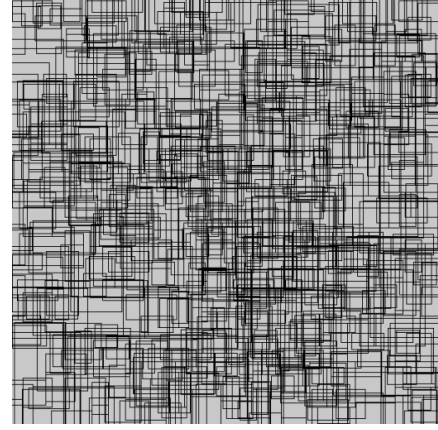


FIGURE 5.6: A visual of all the anchor boxes created during prediction

#### 5.4.4 Intersection over Union (IoU)

We can determine the degree to which the ground-truth (bounding box) and the anchor box are comparable to one another. We are aware that the Jaccard index may be used to quantify the degree to which two sets are same. When sets A and B are considered, the Jaccard index of the sets is calculated by dividing the size of their intersection by the size of their union:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (5.4)$$

The pixel area of any bounding box may be thought of as a set of pixels for our purposes. Using this method, we are able to determine how similar the two bounding



boxes are by comparing the Jaccard index of their respective pixel sets. Jaccard index of two bounding boxes is often referred to as the intersection over union (IoU) index. This is because the IoU index is the ratio of the two boxes' intersection areas to their union areas. An IoU may have a value between 0 and 1, with 0 indicating that two bounding boxes do not overlap at all and 1 indicating that the two bounding boxes are equal. Hence, IoU is a good quantitative measure of the similarity index between two bounding boxes

### 5.4.5 Non-Maximum Suppression

A typical RPN network will have numerous anchor boxes, which will result in the network predicting many bounding boxes that are quite similar to one another for a single object. In order to make the solution more straightforward, we use a method known as Non-Maximum Suppression (NMS) to get rid of all but one of the bounding boxes associated with each prediction.



FIGURE 5.7: Comparison of image showing the reduction of bounding box after applying Non-Maximum Suppression

The underlying algorithm of Non-Maximum suppression is as follows - the object detection model computes the anticipated probability for each class using the predicted bounding box  $B$  that it has predicted. The likelihood that is anticipated to be the highest is denoted by the letter  $p$ , and the class that corresponds to this prediction is the class that is projected to be  $B$ . To be more specific, we refer to the

anticipated bounding box B's confidence (score) as  $p$ . On the same image, each of the projected non-background bounding boxes is ranked in terms of confidence and placed in decreasing order to produce a list denoted as  $L$ . Following that, we will handle the sorted list denoted by the letter  $L$  following these steps:

1. Choose the predicted bounding box  $B_1$  with the greatest level of confidence from set  $L$  to use as the basis. Then, discard all of the non-basis predicted bounding boxes from set  $L$  whose IoU with  $B_1$  is greater than a threshold  $\epsilon$  that has been determined. At this juncture,  $L$  will preserve the projected bounding box that has the greatest level of confidence, but it will get rid of any others which are very similar. Those bounding boxes whose confidence ratings are less than maximum are excluded from the analysis, and hence non-maximum scores get suppressed.
2. Choose the projected bounding box  $B_2$  from  $L$  that has the second greatest level of confidence to use as another basis. Then, eliminate all of the non-basis predicted bounding boxes from  $L$  whose IoU with  $B_2$  has a value that is greater than  $\epsilon$ .
3. Repeat the procedure (step 1 and 2) until all of the bounding boxes that were anticipated for  $L$  have been utilised as a foundation. At the moment, the IoU of any pair of anticipated bounding boxes in  $L$  is lower than the threshold value of  $\epsilon$ ; hence, no pair is too comparable to the other.
4. Output a final list containing all of the bounding boxes that were predicted using the list  $L$ .

#### 5.4.6 Evaluation Metrics

To evaluate the object detection neural network, **Mean Average Precision - mAP** is used as a criterion. **mAP** is based on the following sub-metrics:

- (a) Intersection over Union (IoU)
- (b) Confusion Matrix

(c) Precision

(d) Recall

The sub-metrics are briefly described below. They are already implemented as attributes in `PyTorch`, and hence need not be programmed to be calculated from scratch.

#### 5.4.6.1 Confusion Matrix

The confusion matrix is a representation which underlines various attributes to interpret different metrics. To create a confusion matrix, we need these four attributes:

- **True Positives (TP):** Prediction (P) and Ground Truth (GT) labels match
- **True Negatives (TN):** The Prediction is False and it matches the GT
- **False Positives (FP):** Predicted label is not a part of GT (Type I error)
- **False Negatives (FN):** No prediction, but it is part of GT (Type II error)

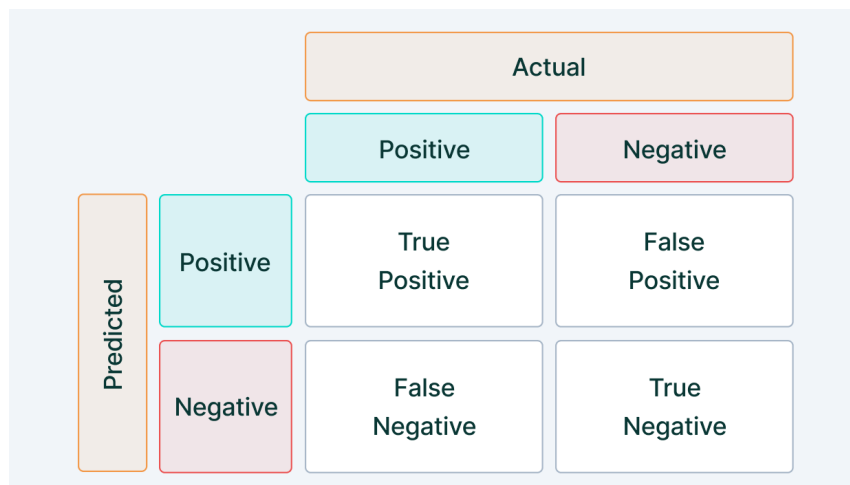


FIGURE 5.8: A visual representation of the confusion matrix

#### 5.4.6.2 Precision and Recall

Precision is the measure of how efficiently did the model find True Positives (TP) out of all Positive Predictions (TP+FP). Now, TP depends on IoU threshold, i.e. if

greater than a IoU, it is a TP else FP. Similarly, Recall measures how good was the model in predicting True Positives (TP) out of all the correct predictions (TP+FN). Mathematically, Precision and Recall are defined

$$Precision = \frac{TP}{TP + FP} \quad (5.5)$$

$$Recall = \frac{TP}{TP + FN} \quad (5.6)$$

#### 5.4.6.3 Mean Average Precision (mAP)

Average Precision, as the name suggest, is the average of precisions as we change thresholds. The weight is the increase in recall from the previous threshold, and the average precision is derived as the weighted mean of precisions at each threshold.

Mathematically, “The mAP is calculated by finding Average Precision(AP) for each class and then average over a number of classes.”

$$mAP = \frac{1}{N} \sum_i^N AP_i \quad (5.7)$$

The **mAP** takes into account both false positives (FP) and false negatives (FN), and accounts for the trade-off between precision and recall (FN). Due to this characteristic, the majority of detecting applications use **mAP** as a accurate metric for evaluation.

### 5.4.7 You Only Look Once

#### 5.4.7.1 Unified Detection

You Only Look Once, abbreviated as YOLO [21] is a technique that is considered to be one of the most cutting-edge and modern approaches to image identification and localisation. The most important benefit that YOLO offers is its rapid processing method, which can complete 45 frames per second and is based on the concept of combining object identification as a single regression task rather than relying on its existing networks. It

unifies the individual tasks of object detection into a single network. In addition, in the process of creating predictions, the YOLO algorithm collects information from an image in a more comprehensive manner than other approaches, such as the Fast-RCNN, which only examines limited regions of an image. Because of this, YOLO is able to make less background errors, which are errors that occur when the object detector mistakes the background for one of the objects it is searching for. YOLO is also capable of learning more generalised representations of things, which makes it a better option for handling inputs that are unexpected or novel.

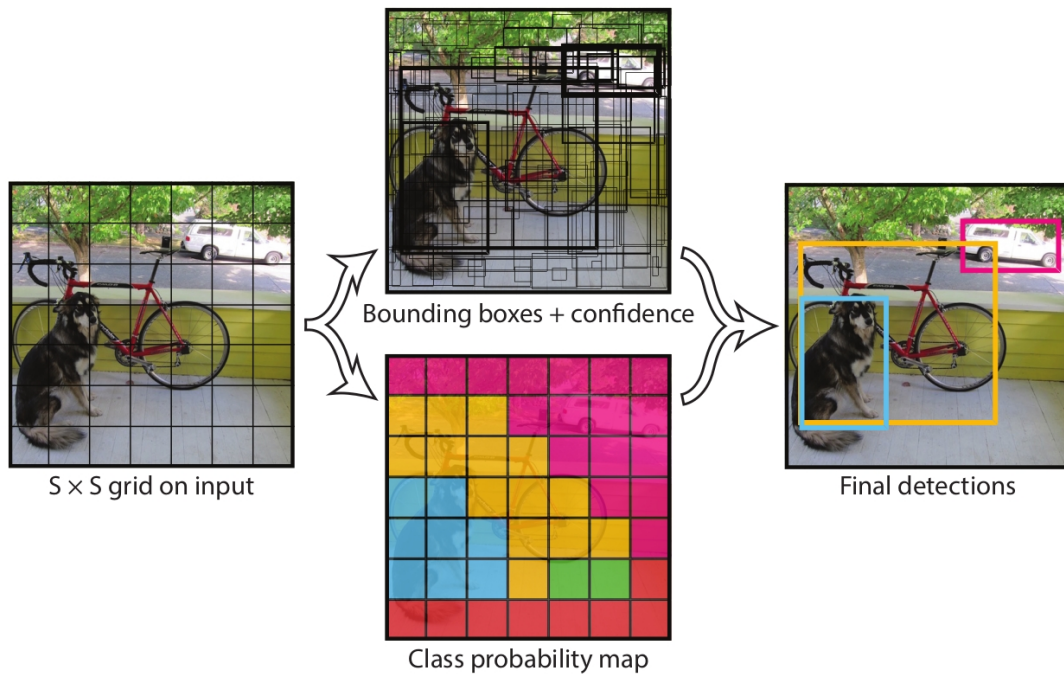


FIGURE 5.9: The description of the unified detection by YOLO family of models. Notice how the class probability map are combined with bounding boxes for final detection

YOLO's functioning theory is based on the idea that it should be able to concurrently predict various bounding boxes and class probabilities for those boxes based on the entire image. This is the goal of the YOLO algorithm. In specifically, YOLO creates a grid that is  $S$  by  $S$  pixels in size from the input images. The  $B$  prior boxes are predicted for each grid cell, along with the confidence score that represents the likelihood of there being an object present in the box. In addition to this, the network makes forecasts on the  $C$  conditional class probabilities for each grid cell, provided that the grid cell in question is

one that includes an object. Finally, a confidence score that is particular to each class is computed for each of the preceding boxes using the following formula:

$$\Pr(C_i | B_j) = \Pr(C_i | \text{Object}) * \Pr(\text{Object}) * IOU_{B_j} \quad (5.8)$$

These scores are a value that stands in for the likelihood that a certain class is included inside a particular box. Additionally, the bounding box offsets are anticipated, which results in bounding boxes that are precise in their fit.

#### 5.4.7.2 Model Architecture

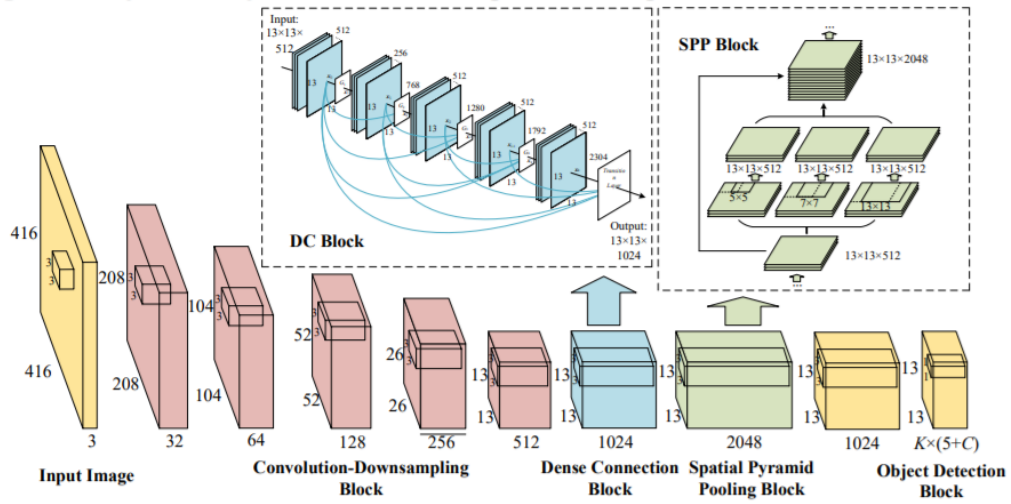


FIGURE 5.10: The backbone network of YOLOv3 depicting spatial pyramid pooling

The design of the network was based on the image-classification network known as GoogLeNet. The YOLO network is comprised of 26 layers total, including 24 convolutional layers and 2 fully connected layers. YOLO[21] essentially employs 1 x 1 reduction layers, followed by 3 x 3 convolutional layers, as an alternative to the inception modules that GoogleNet makes use of. The 7 x 7 x 30 prediction tensor is the output that the network generates after it completes. The first few layers of the network are called the initial convolutional layers, and they are responsible for extracting information from the image. The fully connected layers, on the other hand, anticipate the output probabilities and coordinates.

Though, the backbone network resembles that of YOLOv3 [22] and hence has been shown in Fig.5.10.

The backbone network of YOLOv5 [23] is unavailable since it was developed by ultralytics and available open-sourced. The open-sourced implementation of YOLOv5 has various model version available as shown in Fig. 5.11. The models trained on the repository were available with pre-training weights for Microsoft COCO. The versions available here vary in depth, number of parameters and hence the GFLOPs and inference speed. Thus, choosing the model was dependent as a trade-off between accuracy and inference speed. Since this work just relies on classification and localization while trying to achieve real-time application, YOLOv5s (small) was chosen.

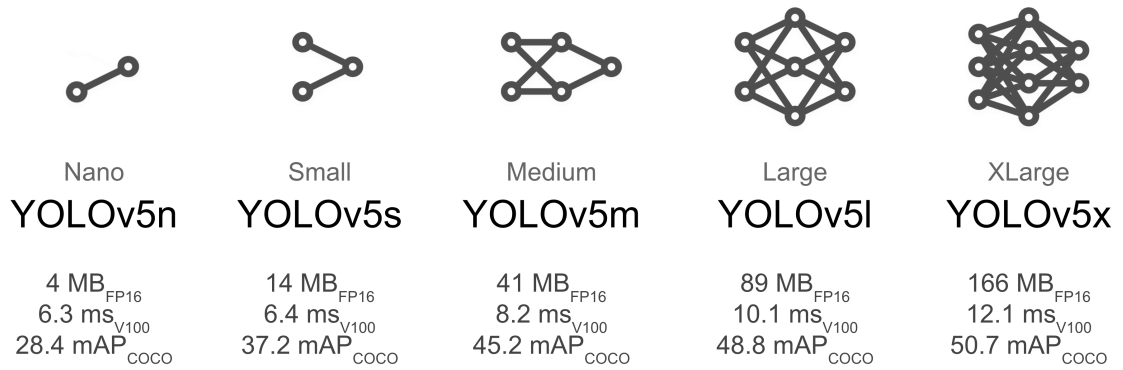


FIGURE 5.11: A comparison of all the available networks for YOLOv5 along with their size and inference time

## Chapter 6

# Methodology and Results

This chapter provides the details associated with the experiments and elaborates the undertaken procedure wherever necessary. Lastly, it concludes with the inferences from those experiments. The various experiments performed have been categorized as shown in Fig. 6.1

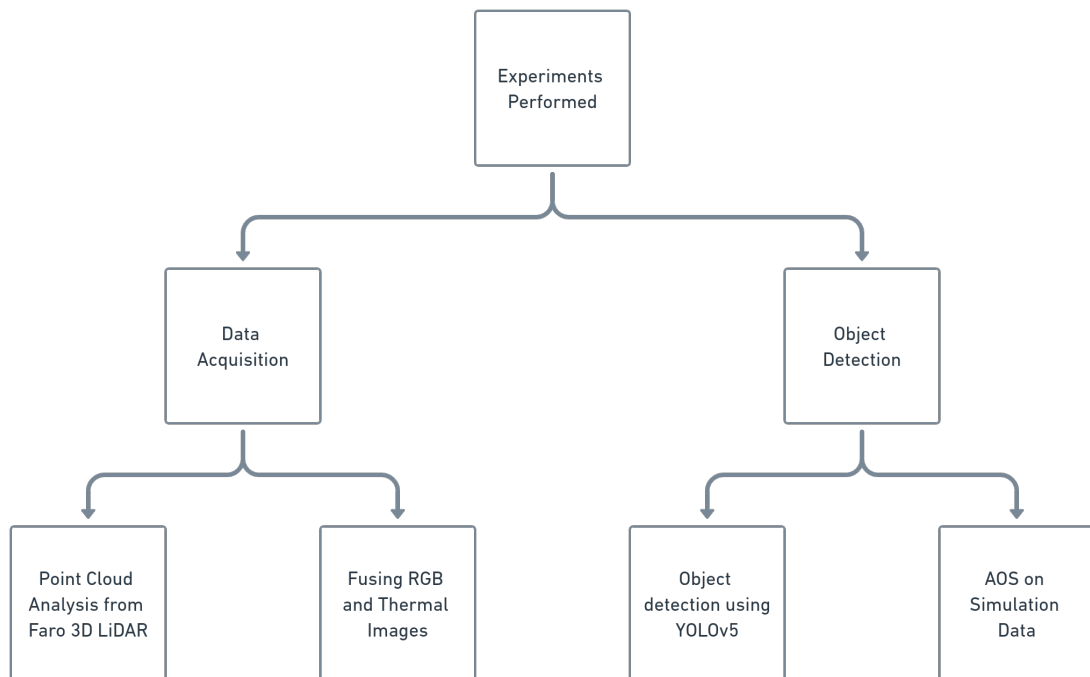


FIGURE 6.1: Flow chart mentioning all the experiments performed



## 6.1 Experiments with Laser Scanner

We conducted several proof-of-concept experiments to determine whether it is feasible to use data from laser scanners. To record scans, the FARO laser scanner was mounted on a fixed tripod. As shown in Fig., a white thermocol rectangle was cut to imitate the object of interest. The white sheet was chosen as an edge case because it reflects the most and if it is not visible, then any other colour object will not be visible either. Consequently, it was an excellent choice for proof-of-concept experiments. There were two scenarios chosen drawing resemblance from varying dense of canopy-

1. The target was concealed behind ground bushes which were  $\sim 30$  cm thick. This is similar to the sparse canopy of vegetation shown in Fig.
2. The scan was taken from a nearby building's terrace, and it resembled a top-down view from a sensor mounted on a UAV flying over a forest canopy. This configuration is depicted in Fig.

As the major objective of this experiment was to determine whether or not there are sufficient feature points to include LiDAR scans into an algorithm, LiDAR motion was excluded. Therefore, LiDAR was put on a tripod and set 30 to 50 metres away from the item. This is comparable to the sensor on a UAV flying at a constant altitude above the ground.

### 6.1.1 LiDAR configuration

The specification of LiDAR are described in the sub-section 3.3. Due to its heavy weight, the LiDAR was designed primarily for ground elevation mapping and was not suitable for remote sensing. Nevertheless, as a proof of concept experiment, the LiDAR serves the purpose of evaluating the number of points in our region of interest. Further, the LiDAR registers a  $360^\circ$  point cloud around itself. Due to the fact that our region of interest is

only in front of the LiDAR, the scan area is restricted as shown in Fig. 6.2 (c) to optimize scan registration time.

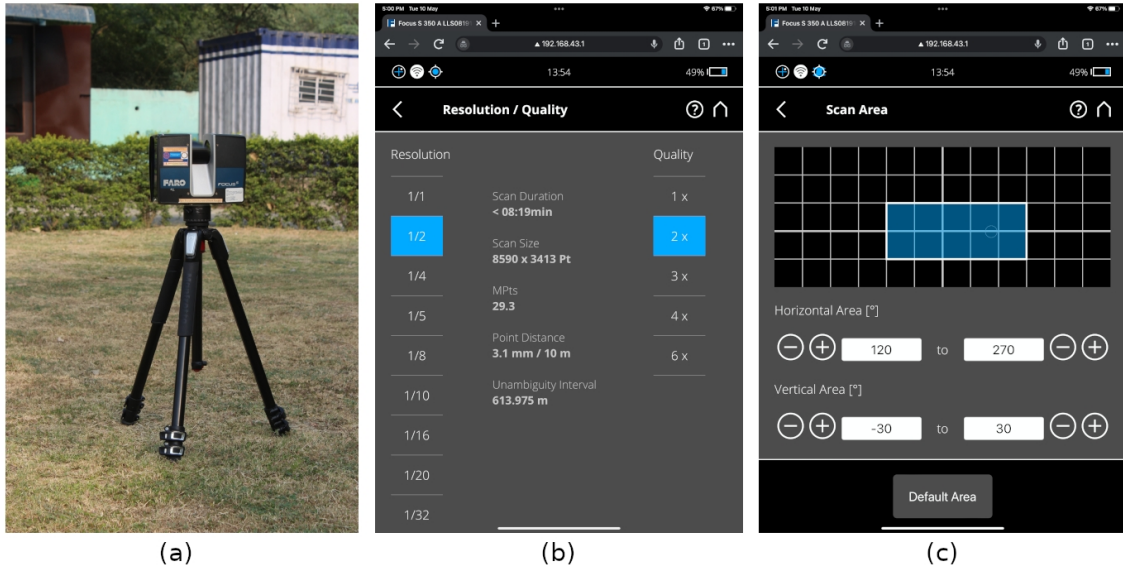


FIGURE 6.2: (a) The LiDAR mounted on the tripod. (b) and (c) The API for LiDAR accessible over port on same network. The panel provides interface for configuring scan meta data.

The parameters used for each experiment are shown in Table 6.1

| Experiment Number | Number of Points | Scan Duration | Resolution | Scan Quality |
|-------------------|------------------|---------------|------------|--------------|
| One               | 29.3 million     | ~9 mins       | 1/2        | 2x           |
| Two               | 9.8 million      | ~7 mins       | 1/2        | 3x           |

TABLE 6.1: The parameters used to register scans for each experiment.

### 6.1.2 Experiment 1

The goal of experiment 1 was to distinguish the point of the object or interest hidden behind vegetation, as described above. As shown in Fig. 6.3, the LiDAR recorded the point cloud, registering 29.3 million points in the region of interest. Because the LiDAR's pitch with respect to the ground frame was  $0^\circ$ , the LiDAR frame has no angular rotation with respect to the ground frame. This circumstance was similar to the partial occlusion in Fig. 6.3 (b).

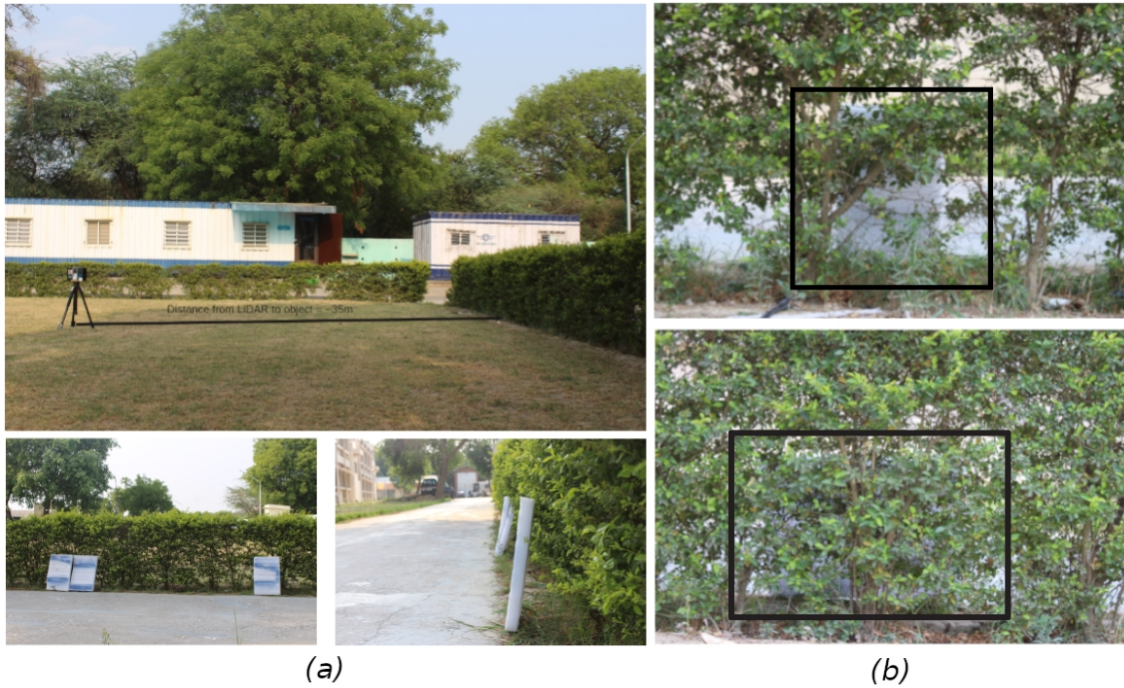


FIGURE 6.3: (a) The LiDAR placed at 35m from vegetation. The side and back view objects were concealed behind bushes. (b) The black box encompass the objects partially visible behind leaves.

#### 6.1.2.1 Observations



FIGURE 6.4: Point cloud of experiment 1, with zoomed region of interest and the overall point cloud on bottom right

There was a dense point cloud, as seen in Fig 6.4., but it couldn't penetrate the leaves. Even if it could, it did not return to the receiver. Additionally, designing an algorithm to



filter  $< 1000$  points from 30 million points would be extremely difficult. As a result, the experiment demonstrated that LiDAR could not map objects obscured by sparse occlusion.

### 6.1.3 Experiment 2

The goal of experiment 2 was to establish the penetration ability of LiDAR beneath the canopy of forest. As a result, as shown in Fig. 6.5 (a), the LiDAR is kept at elevation, with the region of interest centered on the canopy and beneath. The object of interest is lying on the ground to maximize the area of reflection. It is marked inside the black rectangle in Fig. 6.5 (b)

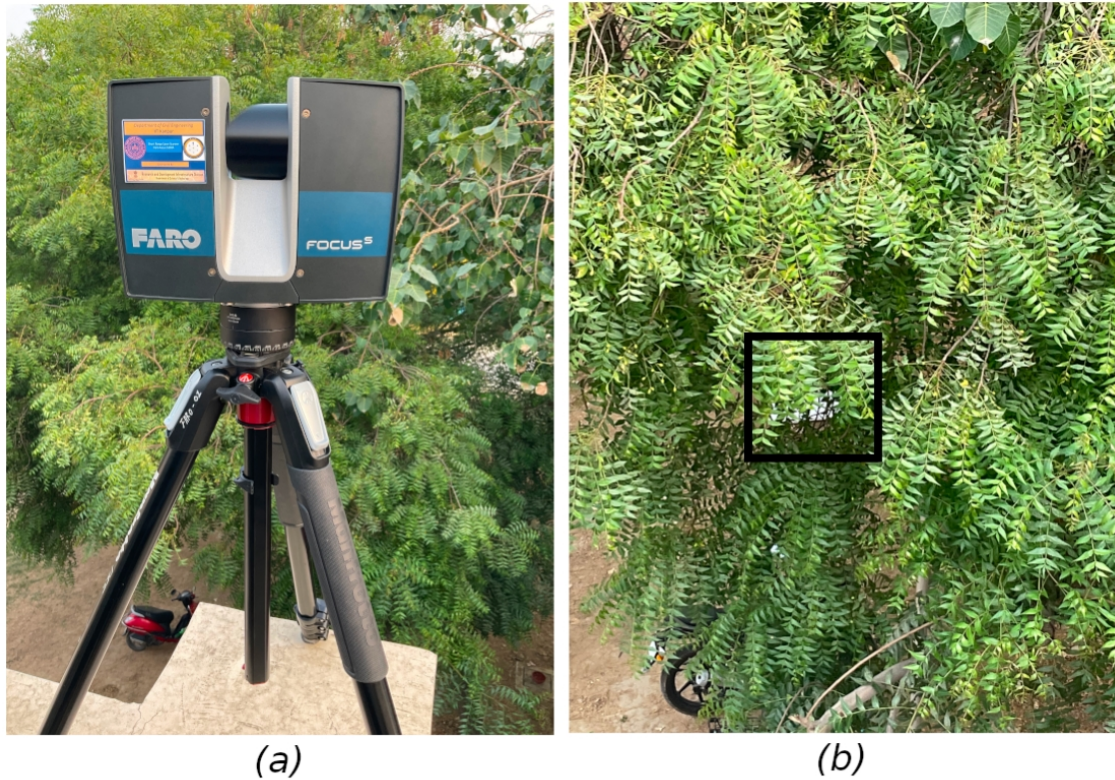


FIGURE 6.5: (a) The LiDAR was installed on top of the building nearby the tree (b) The black box encompass the objects partially visible behind leaves.

### 6.1.3.1 Observations

The point cloud maps non-occluded regions such as the person and scooter, as shown in Fig. 6.6. However, it could only detect the top of the canopy, and penetration below the leaves was too small to be considered significant by any post-processing algorithm. **In conclusion, adding LiDAR would not add to any significant information for the model to learn**



FIGURE 6.6: The point cloud visualized in CloudCompare. It could not register anything below the canopy

## 6.2 RGB and Thermal Fusion

The fusion framework was established from VIFB [24], with code implementation in MATLAB and tuning parameters identical to those mentioned in the original paper. For establishing the pipeline, a few images were taken from the original VIFB framework while three images were used to imitate similar use cases with the thermal and RGB camera.

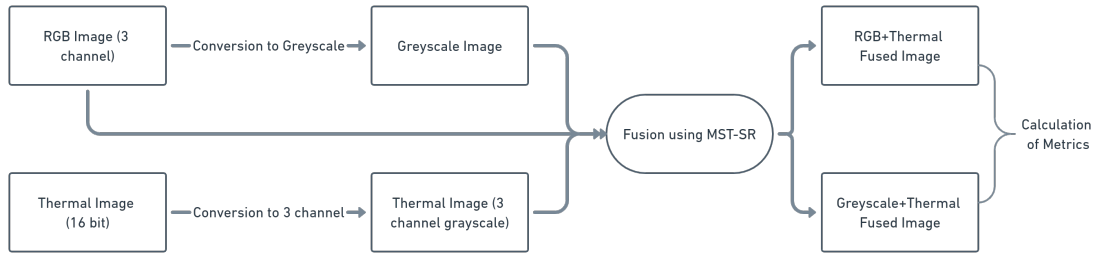


FIGURE 6.7: Process flow of Fusion Framework using MST-SR

### 6.2.1 Results on VIFB images

Two images were chosen, one RGB and the other Greyscale, along with the corresponding IR images. The visual results are shown below, along with some brief observations about them.

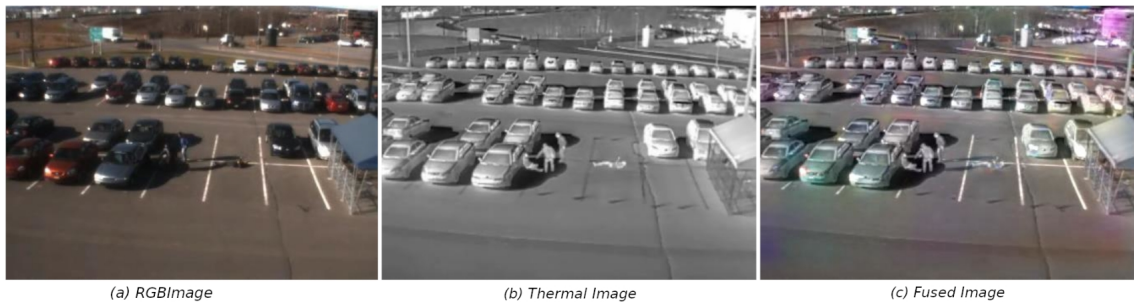


FIGURE 6.8: Result of fusion on the RGB image where few people are hidden inside the shadow of the light source

**Observation:** The RGB image Fig. 6.8 (a) had few people sitting in the shadow of the car and hence were not visible. Even though they were visible in IR images, they blocked out other details because of how they reflected light. Further, a few reflections of light

caused a patch of similar pixels in the IR image. Both the relevant features (humans) and the other portion of the background are free of noise, as seen in the fused images. The only caveat is that using a three-channel RGB image leads to some color perturbations as seen towards top right of the fused image in Fig. 6.8 (c).



FIGURE 6.9: Result of fusion on the greyscale image of a person hidden behind the light source

**Observation:** The greyscale image in Fig. 6.9 (a) has a big patch of white pixels which hide the person behind that lens glare. The person can be seen in the IR image along with other features to help localization. Further, as suspected, the single-channel grey-scale image does not induce any gradients due to multiple-channel input. Thus, it seems better to move forward with greyscale images while finalizing the fusion pipeline.

### 6.2.2 Results on self-acquired images

To test the algorithm's performance on actual self-acquired images, thermal and RGB images were captured using FLIR S60 as described in section 3.2. The white thermocol (as shown in section 6.1 was taken to be the object of interest, and the images were acquired to enhance the features for the same. There were two scenario set up to test the algorithm at the extreme

- Scenario 1 - Unoccluded object with non-contrasting background (sky).
- Scenario 2 - The object is obscured by small bushes to simulate a canopy caused by vegetation.

The following are the results and observations from the experiments:



### 6.2.2.1 Scenario 1

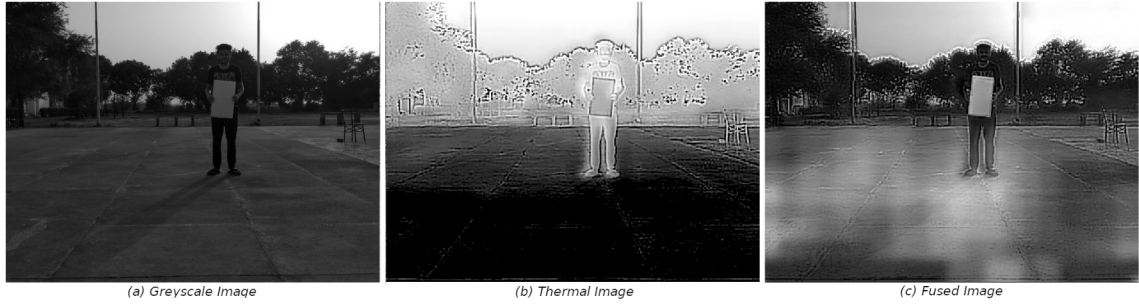


FIGURE 6.10: Result of fusion on the images of a person holding the object behind a non-contrasting background

**Observation:** Firstly, the RGB modalities were converted into grey-scale to inhibit any issues induced due to multi-channel input. As seen in the greyscale image, the background can be distinguished with the object under observation, but the object itself is not very bright. There is no doubt that morphological operations must be used to improve the object's features. As seen in Fig. 6.10 (b), i.e. the thermal image, clearly the object has similar representation as the background and this would result in our detection network performing badly on the given image. Thus, the goal of our fusion algorithm is to improve object visibility while using suitable features from greyscale images and to suppress falsely highlighted background features in IR images. The same has been achieved in Fig. 6.10 (c), where everything except the object is negative. The performance of our network on such image would be correct, since this modality of this image resembles the training set of our network.

### 6.2.2.2 Scenario 2

**Observation:** Again, the RGB modalities were converted to greyscale first. As seen in Fig. 6.11 (a), the object as looks similar to the vegetation. As seen in Fig. 6.11 (b), thermal modality highlights the object but the other features are degenerative. This is largely due to the low resolution and induced noise of IR sensor. This noise would result in unpredictable output from the detection network. Hence, in this case, the fusion network





FIGURE 6.11: Result of fusion on the images of the object behind the occlusion of vegetation

should suppress the noise while still highlighting the object under consideration. The result of the fusion achieves that as seen in Fig. 6.11 (c).

**Remark:** During the experiments, it was also observed that the resolution and noise of the images impacted the final output from the fusion pipeline. Further, the quality of features directly depend on image resolution and less noise. Hence, this is established that better the sensor resolution, better the detection.

### 6.3 Object Detection using YOLOv5

Once the pipeline for fusing thermal and RGB modalities was complete, the next step was to retrain a object detection neural network to accommodate the features representative of thermal modalities. Since the fusion framework associates closely to the modalities of thermal images, the idea was to implement the transfer learning framework on a pre-trained YOLOv5 model as described in section 5.4.

#### 6.3.1 EDA on Teledyne FLIR Dataset

To accomplish the same, an exploratory data analysis (EDA) was necessary to tune the model to the best possible on the training set. The same was performed on the FLIR thermal dataset as mentioned in sub-section 3.4.2. It was found that out of the three

classes: `person`, `bicycle`, and `cars`, there existed a clear imbalance. The distribution of various classes is shown below in Fig.6.12.

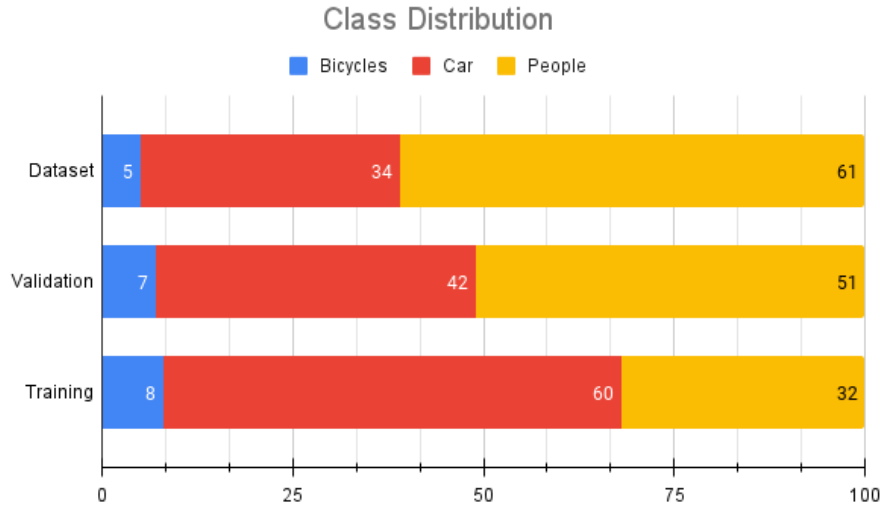


FIGURE 6.12: Bar chart showing the imbalance of classes in FLIR thermal dataset

To solve this issue, the oversampling technique was used. This ensured that the classes with very few samples were exposed again and again to the network. Another technique was applied to expose it to rare classes such as bicycles. To achieve that, same frames were in the class set several times. This improved the `mAP`.

Further, custom anchors were introduced to adapt to the dataset well. Pre-trained weights were taken for COCO dataset, and only last few layers were retrained for the purpose of detection on combined modalities. This gave the following advantages -

- Using custom anchors improved `mAP` scores by detecting small sized object such as person. This is particularly essential because we would like to detect small
- Using pre-trained weights helped network converge faster with less training resources.

### 6.3.2 Inference and Results

The table below shows the theoretical results of model training. Clearly, the most optimized model (last one) achieved the highest `mAP`.

|                                     | mAP on different classes using pyCOCO |             |             |             |          |
|-------------------------------------|---------------------------------------|-------------|-------------|-------------|----------|
| Model                               | People                                | Bicycles    | Cars        | All         | Epochs   |
| YOLOv5 (no pretraining)             | 0.37                                  | 0.43        | 0.72        | 0.51        | 10       |
| YOLOv5 (imbalanced dataset)         | 0.59                                  | 0.14        | 0.86        | 0.47        | 8        |
| YOLOv5 (original anchors)           | 0.62                                  | 0.74        | 0.84        | 0.73        | 6        |
| YOLOv5 (custom anchors)             | 0.75                                  | 0.71        | 0.86        | 0.78        | 6        |
| <b>YOLOv5 (final and optimized)</b> | <b>0.75</b>                           | <b>0.88</b> | <b>0.87</b> | <b>0.83</b> | <b>5</b> |

TABLE 6.2: Result of mAP calculation of FLIR Thermal Dataset

Below are some visual results of model's performance.



FIGURE 6.13: Results of detection on the FLIR test data using YOLOv5. The right-most image is the bounding box created using the thermal modality, whereas the middle image is the matching RGB image. The blue box represents bicycles, the red box represents cars, and the green box represents people.

## 6.4 AOS on Simulation Data

This section describes the Airborne Optical Sectioning experiment on a synthetic dataset generated with the simulator described in section 3.5. The **forest-21** preset was used to generate the data. The experiment’s goal is to see if AOS improves the visibility of artefacts on the ground, if registered in either modality images (RGB and IR).

The steps involved in data generation were as follows.

1. An forest area of  $300\text{m} \times 300\text{m}$  was generated and populated with **forest-21** preset as a model for sparse occlusion. 50 humans were spawned on the ground.
2. A camera, resembling a UAV, was spawned at a height of 30 metres above the ground. The camera captured greyscale and IR image from RGB images using a plugin.
3. Images from both the modalities are converted into 8bit greyscale images and fused using the pipeline described in Section 6.2.
4. The poses of each individual image are saved in JSON files, which are then used for synthetic image rendering.

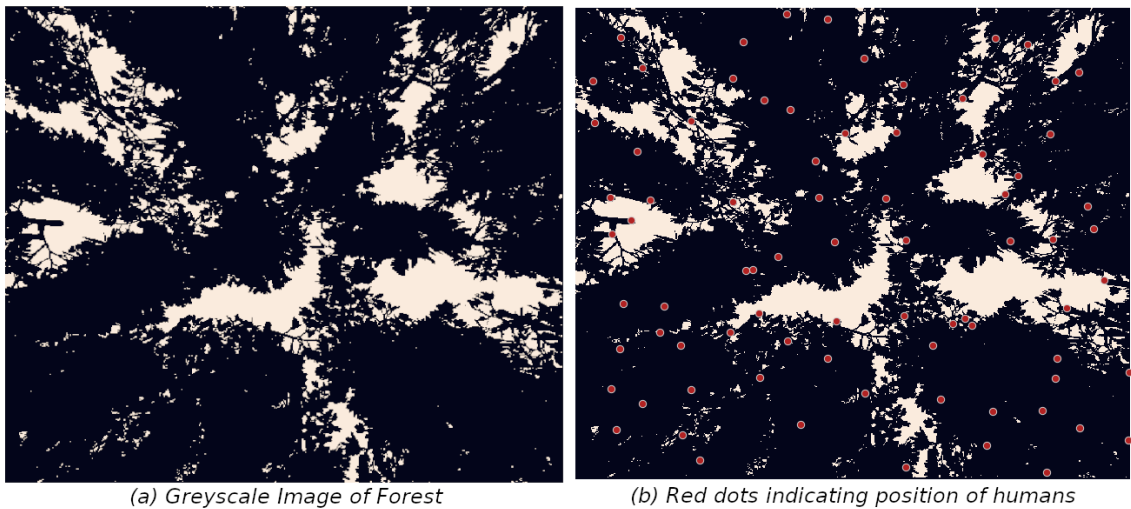
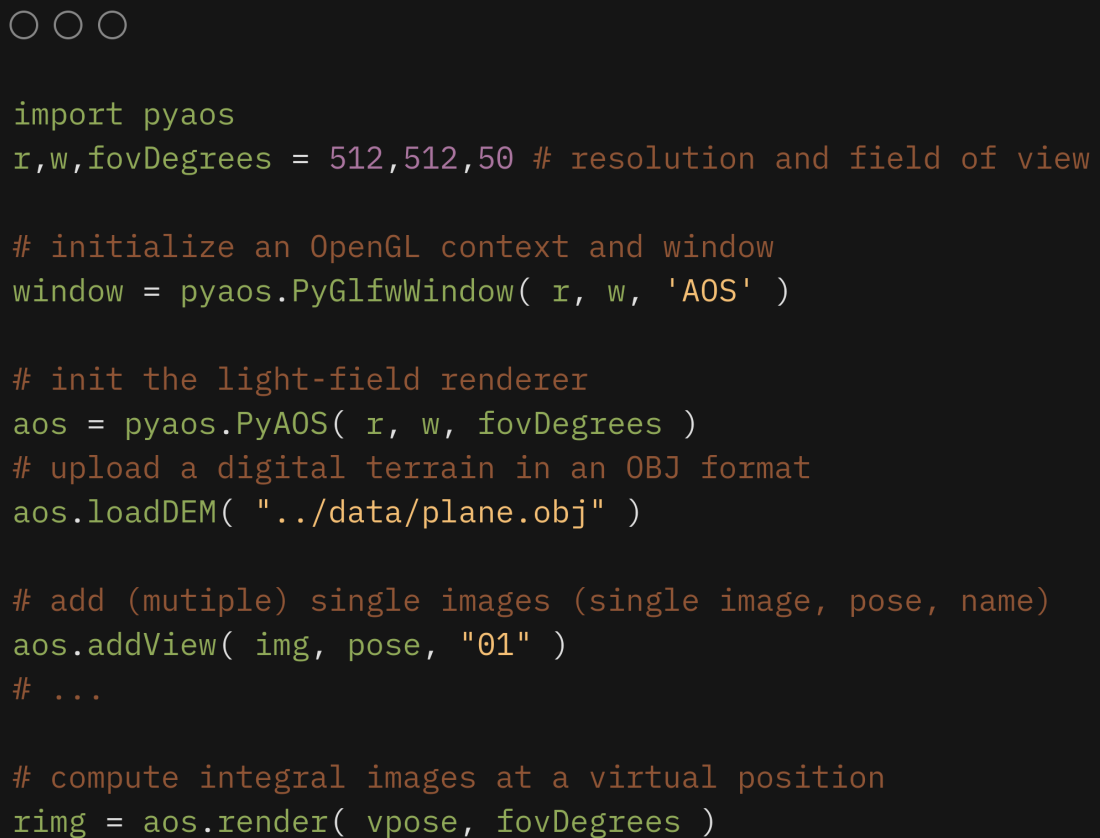


FIGURE 6.14: (a) Top view of the forest. (b) Red dots indicating position of humans

Figure 6.14 shows a visual sample of the rendered images. The images are greyscale, with white representing the visible ground. There are a few red dots that are occluded; some are completely visible while others are completely hidden by the canopy.

After the data from the simulation was generated using the specific procedure mentioned above, it was merged using the LFR algorithm described earlier. Python plugins were used to integrate the images to make the process easier. The script used to perform AOS is shown in Fig 6.15.

A screenshot of a code editor with a dark background and light-colored text. At the top left, there are three small white circles. The code is a Python script for performing AOS (Aperture Opening Simulation). It starts with an import statement for 'pyaos'. Then, it defines variables 'r', 'w', and 'fovDegrees' with values 512, 512, and 50 respectively, with a comment '# resolution and field of view'. Next, it initializes an OpenGL context and window using 'pyaos.PyGlfwWindow'. Then, it initializes a light-field renderer using 'pyaos.PyAOS'. It uploads a digital terrain in OBJ format using 'aos.loadDEM'. It then adds multiple single images using 'aos.addView'. Finally, it computes integral images at a virtual position using 'aos.render'.

```
import pyaos
r,w,fovDegrees = 512,512,50 # resolution and field of view

# initialize an OpenGL context and window
window = pyaos.PyGlfwWindow( r, w, 'AOS' )

# init the light-field renderer
aos = pyaos.PyAOS( r, w, fovDegrees )
# upload a digital terrain in an OBJ format
aos.loadDEM( "../data/plane.obj" )

# add (mutiple) single images (single image, pose, name)
aos.addView( img, pose, "01" )
# ...

# compute integral images at a virtual position
ring = aos.render( vpose, fovDegrees )
```

FIGURE 6.15: Python script to perform AOS

The script necessitates that the user enter the synthetic aperture of the intended integral image output. Then, the LFR module is initialised, and each image and pose can be added to the interface. 15 sequential images were merged to get the final integral image. The incremental results are displayed in ahead.



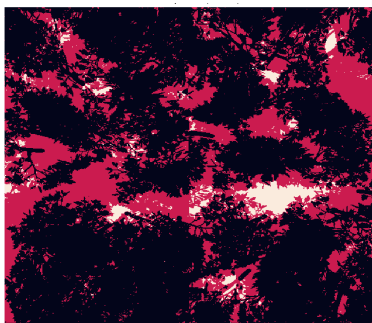


FIGURE 6.16: Image 1

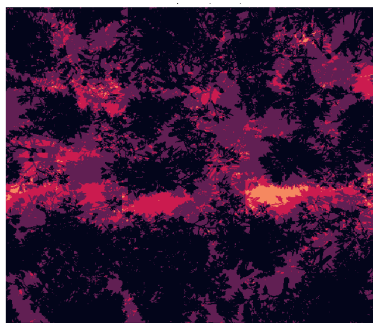


FIGURE 6.17: Image 2

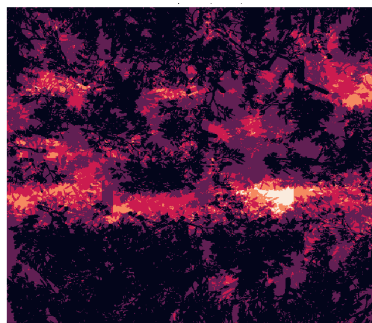


FIGURE 6.18: Image 3

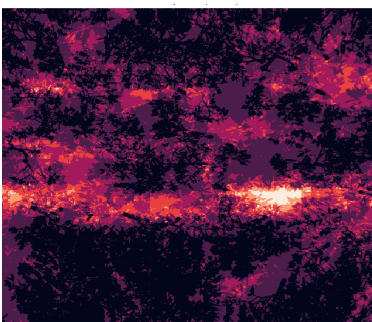


FIGURE 6.19: Image 4

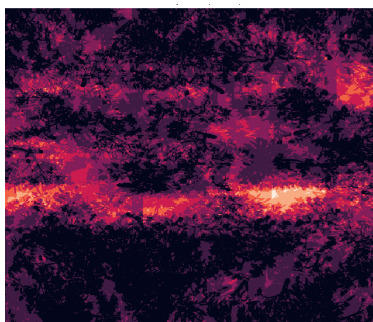


FIGURE 6.20: Image 5

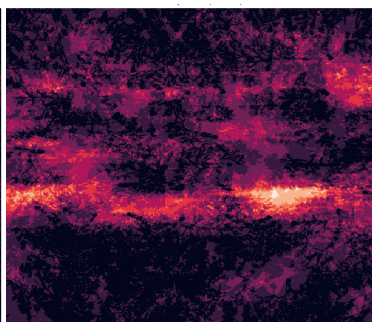


FIGURE 6.21: Image 6

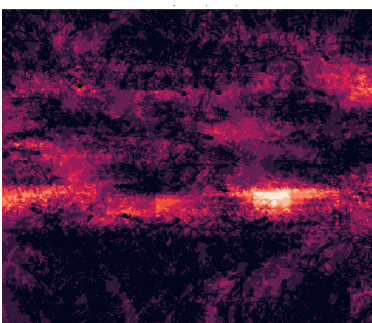


FIGURE 6.22: Image 7

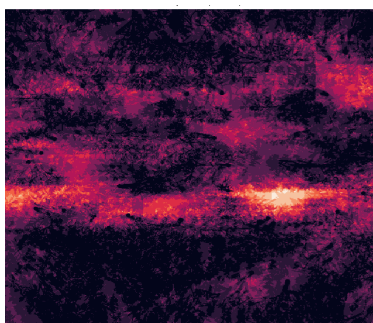


FIGURE 6.23: Image 8

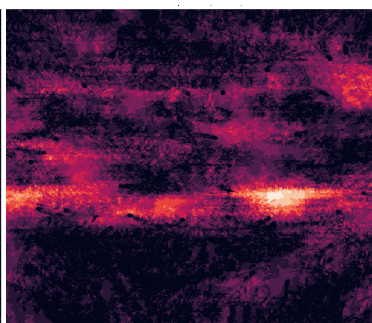
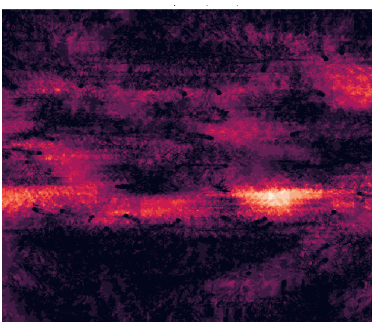
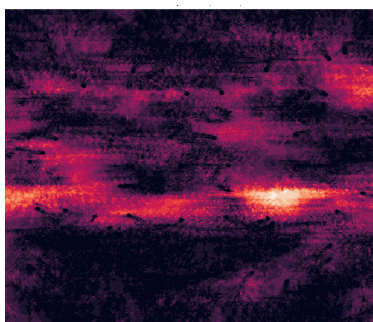
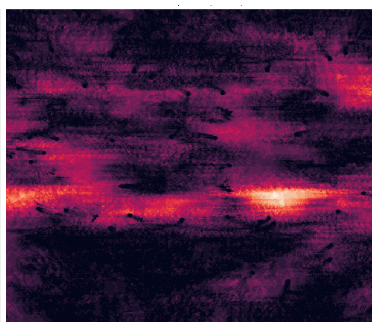
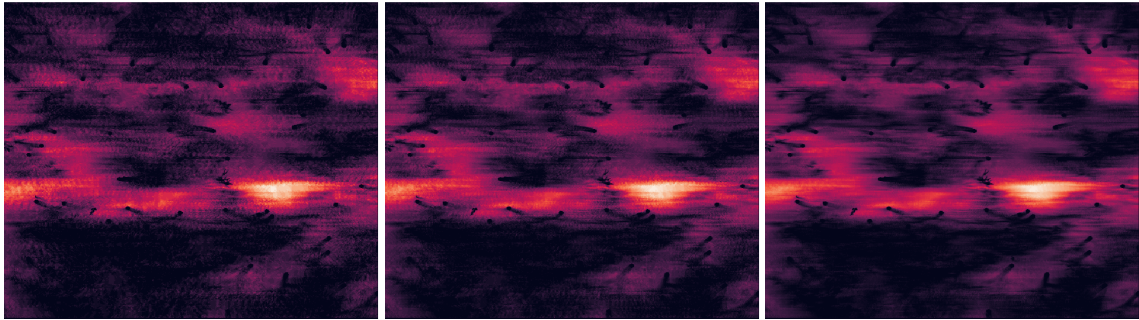


FIGURE 6.24: Image 9

FIGURE 6.25: Image  
10FIGURE 6.26: Image  
11FIGURE 6.27: Image  
12

FIGURE 6.28: Image  
13FIGURE 6.29: Image  
14FIGURE 6.30: Image  
15

From Fig. 6.16 to Fig. 6.30, it is evident that adding an image with a correct pose blurs the leaves and accentuates the partially obscured dots (resembling humans). The final integral image is displayed in 6.31. **The previously obscured black dots are now vividly apparent.** This image may now be forwarded to the YOLOv5 detection network.

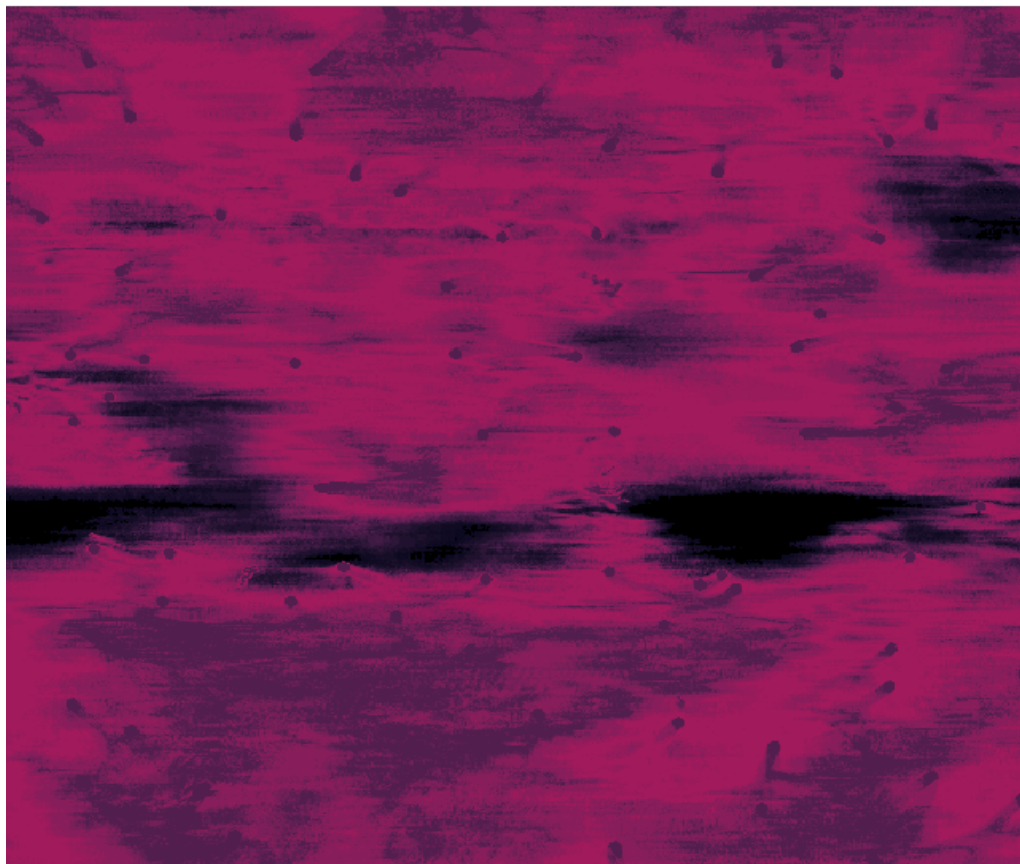


FIGURE 6.31: The final integral image from AOS on synthetic data

## Chapter 7

# Conclusion

This chapter summarizes the work by discussing the relevance and limitations of the thesis's research. In addition, a brief discussion on potential developments that can further improve the work's methodology has been included.

### 7.0.1 Contributions of the Work

We draw parallels from the objectives shown in Fig. 1.1 to highlight the contribution in each of them.

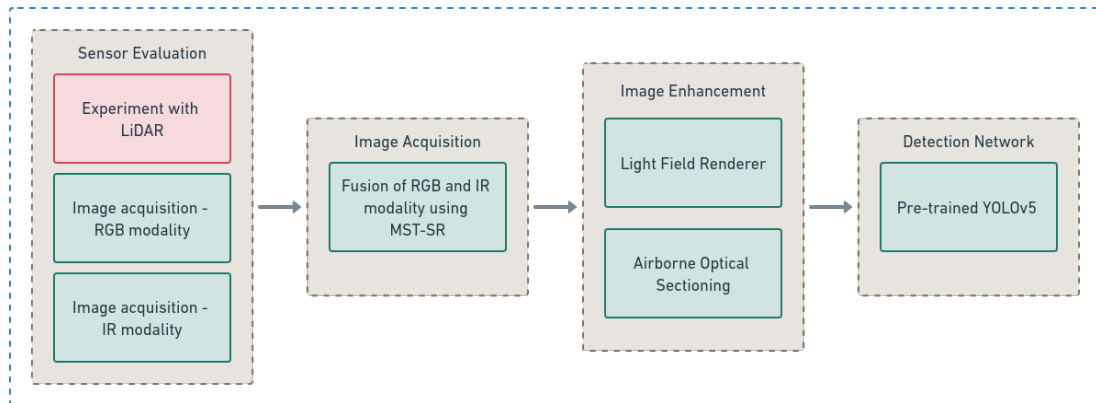


FIGURE 7.1: The significant contribution of this work. Red indicates negative experiments, while green indicates positive experiments.



- Experiments with LiDAR contributed little to the pipeline other than demonstrating that adding LiDAR data into the algorithm would not result in meaningful gains. As an active sensor, LiDAR largely depends on the item of interest's reflection. However, if reflections are obtained, they represent high confidence low noise data. Thus, LiDAR is only helpful when projected on images, given that objects reflect significant points.
- The fusion algorithm enhances the final output by combining RGB and IR spectrum modalities. The fusion pipeline is reasonably stable and is anticipated to operate in close proximity to real time.
- Light Field Rendering also was effective, although the process of generating an integral image is highly dependent on the camera's pose while capturing image. Noting that the photos were rasterized vectors (synthetic data) devoid of noise in the features as well as the poses is important, as the simulation results indicate a significant reduction in vegetation. Consequently, it may be necessary to fine-tune AOS for real-world data and implement noise-removal filters to image poses, given that real world data is susceptible to noise.
- The transfer learning paradigm was effective for training the neural network without requiring voluminous computational resources and data. This ensures that the arduous operation of dataset generation for supervised learning does not impede the development of the model. YOLOv5 trained on thermal modalities performed well and with high confidence to detect objects. The model also performs inference in less than 5 milliseconds, therefore the detection-induced latency is low. Additionally, the model's size has been reduced, making it suitable for embedded deployment.

Lastly, the objective of this research to undertake proof-of-concept experiments in order to lay the groundwork for an end-to-end pipeline has been accomplished.

### 7.0.2 Limitations of the Work

The most significant limitation of this work is the absence of end-to-end pipeline testing with real-world data. In addition, the algorithms perform well when the occlusion is sufficiently sparse, but their performance tends to deteriorate when the occlusion is dense enough to completely block visibility.

### 7.0.3 Future Prospects

Since the methodology described in this study is a preliminary step towards the end goal of an end-to-end pipeline, numerous improvements are conceivable. Some of them could be

- Deployment of large-scale Swarm UAV Systems with many thermal cameras to increase trust in AOS operations.
- Introducing a GAN-based unsupervised learning paradigm to create additional data from limited real-world data samples.

# Bibliography

- [1] G. Contributor, “Next-generation lidar: Seeing the forest through the trees,” Oct 2022.
- [2] D. C. Schedl, I. Kurmi, and O. Bimber, “Search and rescue with airborne optical sectioning,” *Nature Machine Intelligence*, vol. 2, pp. 783–790, nov 2020.
- [3] I. Kurmi, D. C. Schedl, and O. Bimber, “Data: Autonomous Drones for Search and Rescue in Forests,” Dec. 2020. Funding: LIT – Linz Institute of Technology (grant: LIT-2019-8-SEE-114).
- [4] I. Kurmi and O. Bimber, “RGB and Thermal Integral Image dataset for Search and Rescue with Airborne Optical Sectioning.,” Mar. 2022.
- [5] D. Hien, “The modern history of object recognition—infographic,” 2017.
- [6] V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning,” 2016.
- [7] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [8] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, “Microsoft coco: Common objects in context,” 2014.
- [9] “Free teledyne flir thermal dataset for algorithm training.”

- [10] D. C. Schedl, I. Kurmi, and O. Bimber, "Search and rescue with airborne optical sectioning," *Nature Machine Intelligence*, vol. 2, pp. 783–790, nov 2020.
- [11] F. Seits, I. Kurmi, R. J. A. A. Nathan, R. Ortner, and O. Bimber, "On the role of field of view for occlusion removal with airborne optical sectioning," 2022.
- [12] M. Levoy and P. Hanrahan, "Light field rendering," in *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '96, (New York, NY, USA), p. 31–42, Association for Computing Machinery, 1996.
- [13] A. Isaksen, L. McMillan, and S. J. Gortler, "Dynamically reparameterized light fields," in *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '00, (USA), p. 297–306, ACM Press/Addison-Wesley Publishing Co., 2000.
- [14] I. Kurmi, D. C. Schedl, and O. Bimber, "Airborne optical sectioning," *Journal of Imaging*, vol. 4, no. 8, 2018.
- [15] O. Bimber, I. Kurmi, and D. C. Schedl, "Jku-icg/aos: Airborne optical sectioning."
- [16] Z. Liu, E. Blasch, Z. Xue, J. Zhao, R. Laganriere, and W. Wu, "Objective assessment of multiresolution image fusion algorithms for context enhancement in night vision: A comparative study," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, pp. 94–109, 01 2012.
- [17] Y. Liu, S. Liu, and Z. Wang, "A general framework for image fusion based on multi-scale transform and sparse representation," *Information Fusion*, vol. 24, pp. 147–164, 2015.
- [18] Y. Chen and R. S. Blum, "A new automated quality assessment algorithm for image fusion," *Image and Vision Computing*, vol. 27, no. 10, pp. 1421–1432, 2009. Special Section: Computer Vision Methods for Ambient Intelligence.
- [19] H. Chen and P. K. Varshney, "A human perception inspired quality metric for image fusion based on regional information," *Information Fusion*, vol. 8, no. 2, pp. 193–207, 2007. Special Issue on Image Fusion: Advances in the State of the Art.

- 
- [20] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” 2015.
  - [21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 2015.
  - [22] Z. Huang, J. Wang, X. Fu, T. Yu, Y. Guo, and R. Wang, “DC-SPP-YOLO: Dense connection and spatial pyramid pooling based YOLO for object detection,” *Information Sciences*, vol. 522, pp. 241–258, jun 2020.
  - [23] G. Jocher, A. Chaurasia, A. Stoken, J. Borovec, NanoCode012, Y. Kwon, K. Michael, TaoXie, J. Fang, imyhxy, Lorna, Yifu), C. Wong, A. V, D. Montes, Z. Wang, C. Fati, J. Nadar, Laughing, UnglvKitDe, V. Sonck, tkianai, yxNONG, P. Skalski, A. Hogan, D. Nair, M. Strobel, and M. Jain, “ultralytics/yolov5: v7.0 - YOLOv5 SOTA Realtime Instance Segmentation,” Nov. 2022.
  - [24] X. Zhang, P. Ye, and G. Xiao, “Vifb: A visible and infrared image fusion benchmark,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2020.